



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
У НОВОМ САДУ



Момир Станишић

**Употреба приступних кључева
као основе за аутентификацију
и енкрипцију без лозинки**

ЗАВРШНИ РАД
Основне академске студије

Нови Сад, 2026.



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ЗАДАТАК ЗА ЗАВРШНИ РАД

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Момир Станишић	Број индекса:	SV 39/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	проф. др Горан Сладић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:			
- проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Употреба приступних кључева као основе за аутентификацију и енкрипцију без лозинки

ТЕКСТ ЗАДАТКА:

Упознати се са принципима аутентификације засноване на приступним кључевима и FIDO2 радном оквиру. Специфицирати и имплементирати систем који користи приступне кључеве за пријаву без лозинки, врши проверу њиховог порекла и из њих изводи кључеве за шифровање. Извршити тестирање, размотрити ограничења и документовати решење.



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	Монографска документација
Тип записа, ТЗ :	Текстуални штампани материјал
Врста рада, ВР :	Дипломски - бечелор рад
Аутор, АУ :	Момир Станишић
Ментор, МН :	др Горан Сладић, редовни професор
Наслов рада, НР :	Употреба приступних кључева као основе за аутентификацију и енкрипцију без лозинки
Језик публикације, ЈП :	српски / ћирилица
Језик извода, ЈИ :	српски / енглески
Земља публикавања, ЗП :	Република Србија
Уже географско подручје, УГП :	Војводина
Година, ГО :	2026
Издавач, ИЗ :	Ауторски репринт
Место и адреса, МА :	Нови Сад, Трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/цитата/табела/слика/графика/прилога)	7/74/56/5/13/0/0
Научна област, НО :	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :	Приступни кључеви, аутентификација, енкрипција
УДК	
Чува се, ЧУ :	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	Рад приказује имплементацију система аутентификације базираног на приступним кључевима уместо на лозинкама. Рад такође демонстрира атестацију приступних кључева, као и њихово коришћење у енкрипцији података.
Датум прихватања теме, ДП :	
Датум одбране, ДО :	25.09.2026.
Чланови комисије, КО :	Председник: др Милан Стојков, доц., ФТН Нови Сад
	Члан: др Никола Лубурић, ванр. проф., ФТН Нови Сад
	Члан, ментор: др Горан Сладић, ред. проф., ФТН Нови Сад



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	BSc thesis
Author, AU :	Momir Stanišić
Mentor, MN :	Goran Sladić, PhD, full professor
Title, TI :	Use of passkeys as the basis for passwordless authentication and encryption
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian / English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Trg Dositeja Obradovića 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/74/56/5/13/0/0
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Passkeys, authentication, encryption
UC	
Holding data, HD :	The Library of the Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	The thesis presents the implementation of an authentication system based on passkeys instead of passwords. The thesis also demonstrates passkey attestation, as well as the use of passkeys in data encryption.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	25.09.2026.
Defended Board, DB :	President: Milan Stojkov, PhD, assist. prof., FTN Novi Sad Member: Nikola Luburić, PhD, assoc. prof., FTN Novi Sad Member, Mentor: Goran Sladić, PhD, full prof., FTN Novi Sad

Садржај

Увод	1
Теоријске основе.....	5
2.1 Криптографске основе	5
2.2 FIDO2.....	7
2.3 Структуре података коришћене у WebAuthn церемонијама	10
2.4 Резидентни и нерезидентни приступни кључеви	13
2.5 Атестација	14
2.6 Проширења намењена за енкрипцију	15
2.7 Хибридни транспорт	15
Архитектура система.....	17
3.1 Циљеви и захтеви система.....	17
3.2 Три подсистема и веза међу њима	18
3.3 Модел претњи и ограничења система	19
3.4 Коришћене технологије	19
Привезак	21
4.1 Улога подсистема и намерна ограничења.....	21
4.2 Интеграција са системским посредником <i>Credential Manager</i>	21
4.3 Хијерархија кључева, заштита PIN-ом и чување метаподатака.....	23
4.4 Формирање одговора аутентификатора	25
4.5 Атестација кључа из безбедног хардвера.....	25
4.6 Одговор на PRF захтеве	27
4.7 Апликација и додатна очвршћавања	28
4.8 Тестирање и ограничења	28
Креденац.....	29
5.1 Архитектура, API и модел података	29
5.2 Регистрација магичном везом	31
5.3 Провера регистрације и пријаве на серверу.....	32
5.4 Порекло кључа као фактор одлуке о правима	34
5.5 Сесије и заштита веб слоја	34
5.6 Подаци у мировању.....	35
5.7 Отпремање и преузимање датотека	36
5.8 <i>Signal</i> интерфејс.....	36

5.9 Тестирање и ограничења.....	38
Локот.....	39
6.1 Проблем и захтеви.....	39
6.2 Од PRF излаза до кључа сефа.....	39
6.3 Формат сефа и формат документа са тајнама	41
6.4 Шема у TOML запису	42
6.5 Команде и испорука тајни	43
6.6 Ослањање на изворне библиотеке	44
6.7 Аутентификација у хибридном режиму транспорта	45
6.8 Библиотека за JVM и подизање Креденца	45
6.9 Тестирање и ограничења.....	46
Закључак.....	47
Списак слика	49
Списак листинга	51
Списак табела.....	53
Списак коришћених скраћеница	55
Списак коришћених појмова	57
Биографија.....	59
Литература.....	61

Глава 1

Увод

Лозинке су тренутно најраспрострањенији метод аутентификације на интернету [2]. У том приступу, најчешће се од корисника очекује да самостално изабере довољно сложену лозинку, такву да је нападач не може открити погађањем. Потребно је да лозинка буде отпорна не само на погађање употребом грубе силе (енг. *brute force attack*), већ и на софистицираније нападе употребом речника који садрже раније виђене лозинке (енг. *dictionary attack*). Међутим, чак и пажљиво изабрана лозинка задржава фундаменталне мане свих лозинки, независно од њихове комплексности. Лозинке је напорно памтити, што се нарочито односи на комплексне лозинке. У пракси се то често манифестује кроз вишеструку употребу исте лозинке за пријаву на различите сервисе. У истраживању скупова процурелих лозинки утврђено је да између 43% и 51% корисника практикује вишеструку употребу исте лозинке [3].

Преостале мане не тичу се памћења, већ самог поступка пријаве. Прва мана је то што корисник при свакој пријави сервису предаје своју тајну вредност. Овим се отвара могућност да је нападач преваром прибави директно од корисника и касније сам употреби. Друга мана је то што сервис ту тајну вредност мора примити, обрадити и сачувати на начин који онемогућава њено накнадно откривање. У супротном база сервиса постаје извор лозинки свих његових корисника. Због вишеструке употребе, прибављена лозинка нападачу вреди више од налога из ког потиче. Нападач који дође до лозинке добија приступ не само корисничком налогу из ког потиче, већ и свим осталим налозима на којима корисник употребљава ту лозинку. Ово важи без обзира на мере заштите које некомпромитовани сервиси примењују, докле год познавање лозинке сматрају довољним за потпуну аутентификацију корисника.

Превара ове врсте спроводи се нападом пецања (енг. *phishing attack*). Нападач поставља страницу која имитира пријаву на постојећи сервис и наводи корисника да унесе своју лозинку. Успешност таквог напада не зависи од сложености лозинке, већ од уверљивости лажиране странице. Визуелна сличност појединих знакова и речи то често олакшава, нпр. низ слова *rn* тешко се разликује од слова *m*. То омогућава креирање лажних домена попут *rnicrosoft.com* или *grnail.com* који на први поглед делују веродостојно. Увођење једнократних кодова (енг. *One-Time Password*) као другог фактора аутентификације овај проблем ублажава, али га не отклања. Унос ОТП кода

није ограничен само на прави домен сервиса, те га лажна страница може проследити нападачу пре истека важења и тако му омогућити пун приступ налогу [4].

Приступни кључеви (енг. *passkeys*) наведене мане потпуно уклањају изменом самог поступка пријаве. Уместо тајне коју корисник предаје, за сваки сервис се формира пар криптографских кључева. Сервис чува јавни кључ, док приватни кључ остаје у аутентификатору. У питању је уређај или програм који креира кључеве, чува их и користи за потписивање. Пријава се спроводи потписивањем насумичног изазова који шаље сервис. Захтев за потписивање садржи и домен странице (енг. *domain*), при чему прегледач (енг. *browser*) проверава да ли се домен подудара са адресом са које захтев потиче, а аутентификатор бира кључ формиран искључиво за тај домен. На овај начин претходно описан напад пецања престаје да функционише. Овакав метод аутентификације стандардизован је FIDO2 радним оквиром (енг. *framework*) у сарадњи FIDO (*Fast IDentity Online*) алијансе и W3C (*World Wide Web Consortium*). Овај радни оквир чине WebAuthn (*Web Authentication*) спецификација [5], као интерфејс између прегледача, странице и сервиса, и СТАР (*Client to Authenticator Protocol*) [6], као интерфејс између прегледача и аутентификатора. FIDO2 је јавно представљен 2018. године, када је WebAuthn достигао статус кандидата за препоруку [7], а сама спецификација постала је препорука конзорцијума W3C у марту 2019. године [8]. Тренутно FIDO2 подржавају водећи оперативни системи, али шире прихватање међу сервисима и корисницима још увек траје [2].

Поред аутентификације, FIDO2 предвиђа и две додатне могућности које чине предмет овог рада. Прва је атестација (енг. *attestation*), потписана тврдња коју аутентификатор прилаже уз креирани јавни кључ и којом потврђује какав уређај је кључ створио, као и под каквим условима га чува [5]. Друга могућност је проширење (енг. *extension*) којим аутентификатор, поред потписивања, из задате улазне вредности и тајне везане за кључ креира псеудо-насумичну вредност. Резултат је увек исти за исти приступни кључ и исту улазну вредност, чиме приступни кључ постаје такође и извор кључева за симетричну енкрипцију произвољних података. Ово проширење СТАР дефинише под називом *hmac-secret* [6], а WebAuthn под називом *prf* [5].

Предмет овог рада јесте систем Јанус, назван по римском богу врата и пролаза. У овом систему су поред аутентификације примењене и две претходно наведене могућности које приступни кључеви пружају. Систем чине три програма. Први је Привезак, менаџер приступних кључева за *Android* платформу, који приватне кључеве чува у безбедном хардверу уређаја. Уз регистрацију нових кључева Привезак такође пружа и атестацију. Други је Креденац, веб апликација за складиштење белешки и датотека, чији се систем за аутентификацију базира на приступним кључевима. Креденац демонстрира употребу атестације тако што функционалности

чувања и приступа датотекама омогућава само корисницима пријављеним помоћу Привеска. Корисници пријављени коришћењем других аутентификатора бивају ограничени на рад са белешкама. Све информације које могу идентификовати корисника (енг. *Personally Identifiable Information*, ПИ), као и његове белешке и датотеке, складиште се на диску искључиво у шифрованом облику. Кључеви за шифровање ових података се такође никад не чувају на диску у отвореном облику, већ се за њихово складиштење користи Локот, трећи део Јанус пројекта. Локот је програм за читање и чување серверских тајни на диску у шифрованом облику, а за њихово дешифровање уместо лозинке користи приступне кључеве.

Рад има три циља. Први је приказ комплетног поступка аутентификације уз помоћ приступних кључева, од стварања кључа у безбедном хардверу уређаја до успостављања сесије у прегледачу. Други је демонстрација атестације као основе за доношење одлуке о нивоу приступа, при чему су покривене обе стране атестације, Привезак који тврдњу формира и Креденац који је проверава. Трећи је демонстрација употребе приступних кључева у сврхе шифровања.

У другој глави изложене су теоријске основе, од основних криптографских функција до WebAuthn и СТАР спецификација на којима се систем заснива. Трећа глава садржи захтеве система, архитектуру, модел претњи и коришћене технологије. Четврта, пета и шеста глава описују редом Привезак, Креденац и Локот, од усвојених решења, преко формата података, до ограничења сваког од њих. Седма глава садржи закључак са правцима даљег развоја.

Глава 2

Теоријске основе

У овом поглављу изложене су основе на којима систем почива. Најпре се описују криптографске функције које се у раду користе, затим FIDO2 радни оквир и две церемоније које дефинише, као и структуре података које се приликом њих размењују. Потом се објашњава разлика између резидентних и нерезидентних приступних кључева, затим атестација и проширење за извођење псеудо-насумичних вредности, а на крају начин на који се пријава спроводи са другог уређаја.

2.1 Криптографске основе

Хеш функција (енг. *hash function*) пресликава улаз произвољне дужине у низ бајтова утврђене дужине који се назива хеш вредност или сажетак [9]. Од криптографске хеш функције траже се три својства. Прво је једносмерност, што значи да за задату хеш вредност није могуће пронаћи улаз који је креира. Друго је отпорност на проналажење другог улаза који резултује истом хеш вредношћу као већ познати улаз. Треће је отпорност на колизије, односно на проналажење било која два различита улаза са истом хеш вредношћу [10]. Систем описан у раду користи функције SHA-256 и SHA-512 (*Secure Hash Algorithm*) [11][12], које дају редом 32 и 64 бајта. Хеш вредност се у наставку појављује у две улоге, као сажетак који се потписује уместо целе поруке и као градивни елемент осталих функција у овом одељку.

НМАС (*Hash-based Message Authentication Code*) је функција базирана на произвољној хеш функцији H која уз поруку M користи такође и тајни кључ K . Резултат је ознака аутентичности, која истовремено потврђује да порука није измењена и да ју је формирала страна која зна тајни кључ. Кључ се у два узастопна хеширања комбинује са две међусобно различите константе *ipad* и *opad* [13][14]:

$$\text{НМАС}(K, M) = H\left((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M)\right)$$

У изразу $\oplus$ означава операцију искључиво или, а $\parallel$ спајање низова бајтова. Овако конструисана функција спречава напад продужењем поруке (енг. *length extension attack*), који би био могућ ако би се пре хеширања порука једноставно надовезала на кључ. Таква рањивост постоји код неких хеш функција из SHA-2 породице [15], укључујући и функцију SHA-256, која се користи у овом раду.

PBKDF2 (*Password-Based Key Derivation Function 2*) служи за извођење кључа из тајне вредности ниске ентропије као што је лозинка или PIN (*Personal Identification Number*) [16]. За извођење кључа користи се псеудо-насумична функција као што је HMAC. Улаз у функцију су лозинка и насумично одабрана вредност која се назива со (енг. *salt*). Како би се израчунао кључ, HMAC функцију је потребно поновити велики број пута, повећавајући тако цену рачунања сваког појединачног покушаја и отежавајући нападе грубом силом.

Симетрична енкрипција подразумева да се исти кључ користи и за шифровање и за дешифровање. У овом раду се користи AES (*Advanced Encryption Standard*) [17], блоковска шифра која обрађује блокове од 16 бајтова и прихвата кључеве дужине 16, 24 или 32 бајта [17][18]. Блоковска шифра сама по себи шифрује тачно један блок, па је за поруке произвољне дужине потребан режим рада који одређује на који начин се узастопни блокови обрађују. Систем описан у раду користи кључеве од 32 бајта.

У раду се користи AES у GCM (*Galois/Counter Mode*) режиму [19]. Овај режим уз шифровање података обезбеђује и потврду њихове аутентичности. Поред кључа и отвореног текста, у GCM такође улази и вектор иницијализације (енг. *Initialization Vector*, IV), вредност која се уз исти кључ не сме користити више пута. Излаз функције јесу шифрат и ознака аутентичности. GCM режим може да прихвати и придружене податке (енг. *associated data*), ови подаци се не шифрују, али се урачунавају у ознаку аутентичности [20]. Измена било ког од ових делова, шифрата, вектора иницијализације, придружених података или ознаке аутентичности, уместо невалидног отвореног текста резултује неуспехом.

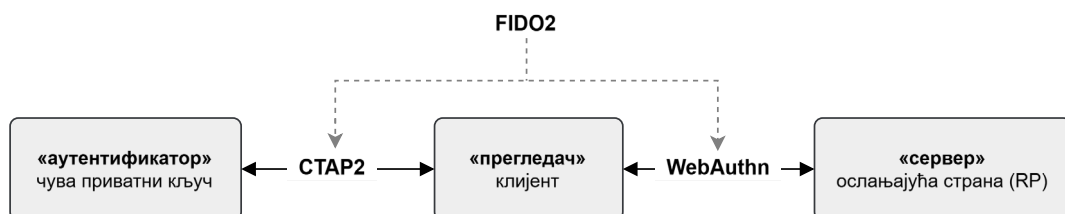
За разлику од симетричне криптографије, где исти кључ служи и за шифровање и за дешифровање, асиметрична користи два различита кључа, јавни и приватни. Кључеви су међусобно повезани тако да из јавног није могуће израчунати приватни. Такав пар кључева може служити за шифровање, али и за дигитални потпис, што је у овом раду од значаја. Приватним кључем се потписује сажетак поруке, док се потпис проверава уз помоћ јавног кључа [21]. Потпис потврђује да порука није измењена и да ју је потписао власник приватног кључа. За разлику од ознаке аутентичности, провера не захтева ниједну тајну вредност, па је може спровести било ко са јавним кључем. У раду се користи ECDSA (*Elliptic Curve Digital Signature Algorithm*) над кривом P-256 [22][23]. Реч је о алгоритму намењеном искључиво потписивању, код ког јавни кључ чини пар координата по 32 бајта, а потпис две вредности исте дужине.

Дигитални потпис сам по себи не говори коме припада јавни кључ којим се потпис проверава. Тај проблем решава сертификат, запис који повезује јавни кључ са идентитетом његовог власника и који је потписан приватним кључем издаваоца сертификата (енг. *Certificate Authority*, CA). Формат таквих сертификата прописује

стандард X.509 [24]. Како и издавалац има свој сертификат, креира се ланац који се завршава коренским сертификатом, самостално потписаним од стране коренског издаваоца (енг. *Root Certification Authority*, Root CA). Подразумева се да је јавни кључ коренског издаваоца унапред познат страни која спроводи проверу потписа. Провера потписа се онда не ослања на поверење у сваки појединачни сертификат, већ само у коренски, док се остали сертификати прихватају на основу непрекинутог низа потписа до коренског. Такав начин организације назива се инфраструктура јавних кључева (енг. *Public Key Infrastructure*, PKI).

2.2 FIDO2

FIDO2 је радни оквир који поступак пријаве приступним кључем дели на два дела, према томе ко са ким размењује поруке. WebAuthn је програмски интерфејс који страница користи да од прегледача затражи стварање новог кључа или потпис постојећим [5]. CTAP је протокол којим прегледач разговара са аутентификатором. Аутентификатор при томе може бити уграђен у сам уређај или прикључен преко USB (*Universal Serial Bus*) порта, бежичне везе или се налази на другом уређају [6]. Подела радног оквира приказана је на слици 1.

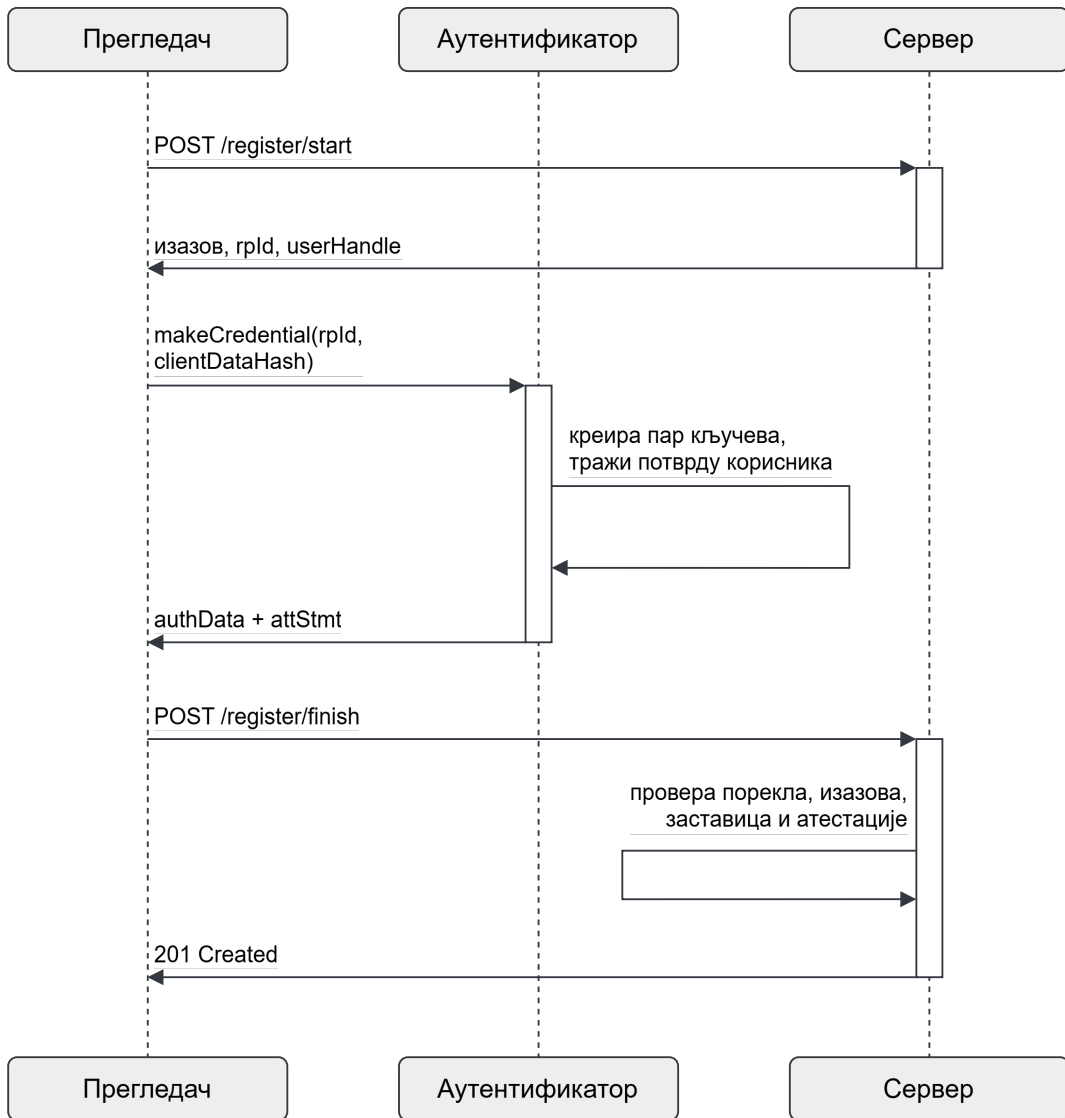


Слика 1: Подела FIDO2 радног оквира на WebAuthn и CTAP протоколе.

У поступку учествују четири стране: корисник који се пријављује, аутентификатор који чува приватни кључ за потписивање, клијентска апликација која посредује (најчешће прегледач) и сервис који захтева пријаву и проверава одговор. Спецификација сервис назива ослањајућом страном (енг. *Relying Party*, RP), а сваки приступни кључ везан је за припадајући домен, који се назива идентификатором сервиса, односно идентификатором ослањајуће стране (енг. *Relying Party ID*, RP ID).

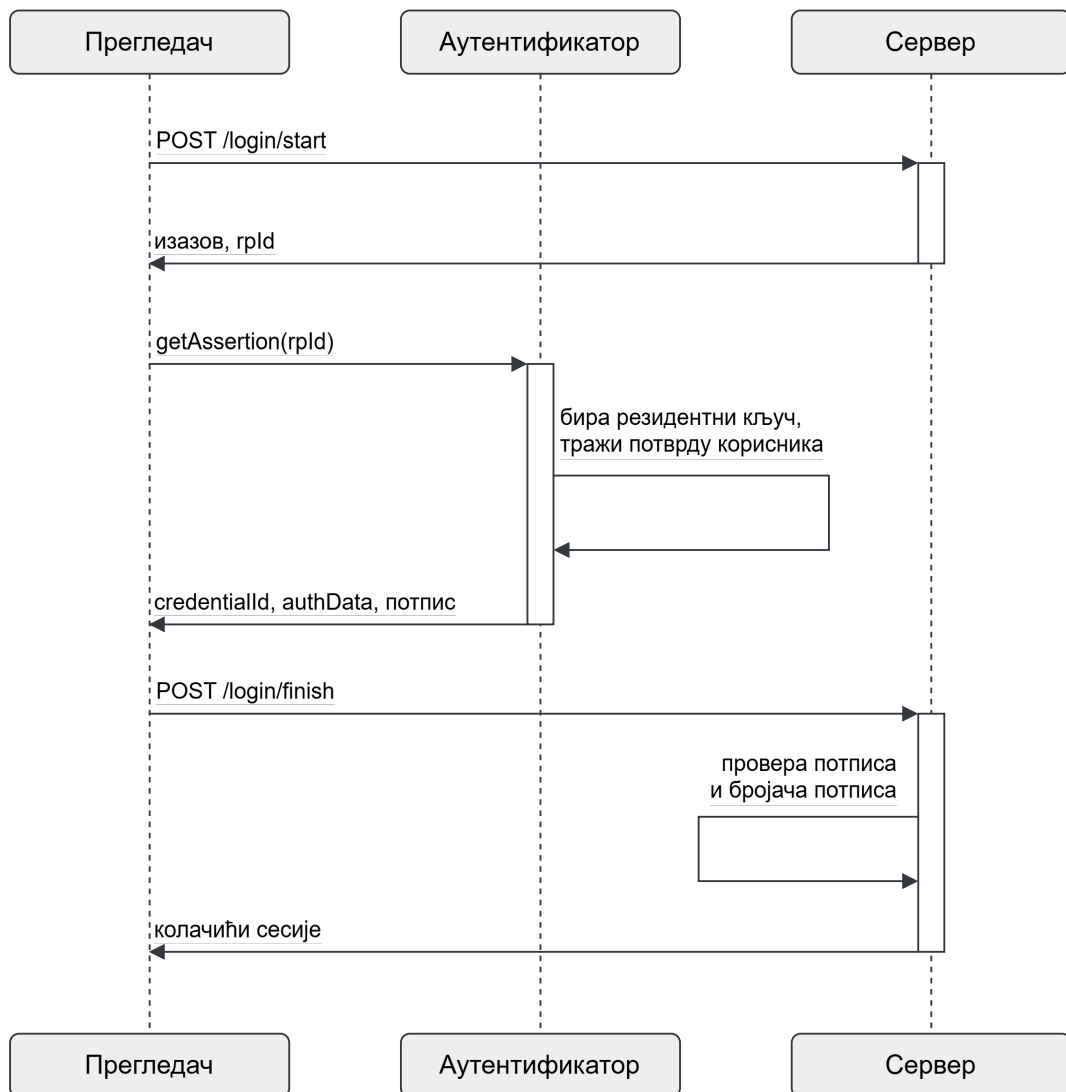
Обе церемоније имају сличан костур. Сервис шаље изазов, насумичну вредност која важи само за то извођење церемоније, спречавајући поновну употребу раније снимљеног одговора. Порекло странице не уписује сама страница, већ прегледач који га заједно са изазовом смешта у податке које предаје даље. Аутентификатор затим тражи потврду од корисника, након чега формира одговор. Одговор се шаље назад сервису који врши његову проверу.

Приликом регистрације аутентификатор ствара нови пар кључева, намењен само том сервису. Приватни кључ задржава, а јавни враћа сервису заједно са идентификатором креираног креденцијала (енг. *credential ID*). Сервис оба податка повезује и чува уз кориснички налог. Приликом пријаве сервис на основу идентификатора проналази одговарајући креденцијал, а потом јавним кључем проверава достављени потпис. Ток регистрације приказан је на слици 2.



Слика 2: Ток церемоније регистрације.

Аутентификатор приликом пријаве бира између сачуваних кључева онај који одговара идентификатору сервиса. Затим саставља запис аутентификатора (енг. *authenticator data*). У њему се налази ком сервису је одговор намењен, колика је вредност бројача употребе кључа и да ли је корисник потврдио присуство и идентитет. Тај запис, заједно са сажетком података клијента (енг. *client data hash*), потписује изабраним кључем. Подаци клијента садрже насумичан изазов и порекло странице, а потпис покрива и њих. Одговор је зато употребљив само за то извођење пријаве и само на домену са ког је пријава затражена. Ток пријаве приказан је на слици 3.



Слика 3: Ток церемоније пријаве.

2.3 Структуре података коришћене у WebAuthn церемонијама

У церемонијама регистрације и пријаве из претходног одељка, клијентска апликација враћа страници податке везане за креденцијал као `PublicKeyCredential` објекат. Креденцијал чини пар кључева које аутентификатор ствара приликом захтева за регистрацију, заједно са припадајућим подацима као што је његов идентификатор. Тај идентификатор налази се у два облика, као низ бајтова у пољу `rawId` и као његов *base64url* запис [25] у пољу `id`, који је погодан за пренос и поређење у текстуалном облику. Објекат садржи и поље `type` са константном вредношћу *public-key*, као и резултате проширења. Остали подаци налазе се у пољу `response`, а његов садржај зависи од типа церемоније. Током регистрације то је `AuthenticatorAttestationResponse` објекат, а приликом пријаве је `AuthenticatorAssertionResponse` објекат [1]. Шеме ових објеката су приказане у листингу 1.

```
interface PublicKeyCredential {
    USVString id; // идентификатор у base64url облику
    ArrayBuffer rawId; // исти идентификатор у бајтовима
    DOMString type; // увек "public-key"
    DOMString? authenticatorAttachment; // "platform" или "cross-platform"
    AuthenticatorResponse response; // садржај зависи од церемоније
    AuthenticationExtensionsClientOutputs getClientExtensionResults();
};

interface AuthenticatorAttestationResponse { // враћа се при регистрацији
    ArrayBuffer clientDataJSON; // подаци клијента, текст у JSON запису
    ArrayBuffer attestationObject; // CBOR мапа са fmt, attStmt и authData
};

interface AuthenticatorAssertionResponse { // враћа се при пријави
    ArrayBuffer clientDataJSON;
    ArrayBuffer authenticatorData; // исти запис као у attestationObject
    ArrayBuffer signature;
    ArrayBuffer userHandle;
};
```

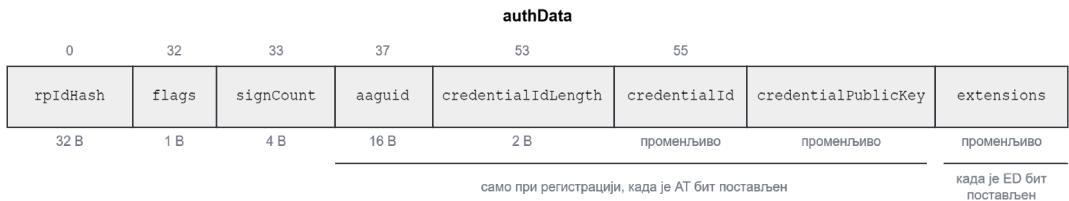
Листинг 1: Објекти које WebAuthn церемоније враћају.

Подаци клијента, у пољу `clientDataJSON`, записани су у JSON (*JavaScript Object Notation*) формату који клијентска апликација саставља пре него што захтев проследи аутентификатору. У њему се налази врста церемоније, изазов који је сервис послао и порекло странице са које је захтев упућен. Поље такође садржи податак о томе да ли захтев потиче из угњежженог оквира чије се порекло разликује од порекла главне странице (енг. *cross-origin iframe*) [5]. Сервис приликом провере сва та поља пореди са оним што је послао и што очекује назад. Изглед записа приказан је у листингу 2.

```
{
  "type": "webauthn.get",
  "challenge": "0KHQotCj0JTQldCd0KLQmCDQn9Ce0JHQLdCC0KPQiNCj",
  "origin": "https://example.com",
  "crossOrigin": false
}
```

Листинг 2: Пример записа clientDataJSON.

Запис `authData`, који аутентификатор саставља и потписује, јесте низ бајтова са утврђеним распоредом поља [5][1]. Прво поље је `rpIdHash`, хеш идентификатора сервиса, по ком сервис утврђује да је одговор био намењен управо њему. Друго поље је бајт са заставицама (енг. *flags*), где поједини битови сигнализирају присуство одређених поља у `authData` запису. Треће поље је `signCount`, бројач употребе кључа. Ако је у питању церемонија регистрације, након њих налазе се и поља са описом креираног креденцијала: `aaguid` (*Authenticator Attestation Globally Unique Identifier*), `credentialIdLength`, `credentialId` и `credentialPublicKey`. При томе `credentialId` садржи исту вредност као поља `id` и `rawId`. Вредност `aaguid` означава произвођача и модел аутентификатора и иста је за све примерке [5][26]. На крају записа, ако постоје, налазе се резултати проширења. Распоред и дужине поља приказани су на слици 4.



Слика 4: Структура `authData` записа.

Бајт са заставицама садржи осам битова, од којих је шест дефинисано, а два су резервисана за будућу употребу [5]. Прва два бита односе се на корисника. Присуство корисника (енг. *User Present*, UP) потврђује да је корисник над аутентификатором извршио радњу као што је додир, а верификација корисника (енг. *User Verified*, UV) да је при томе проверен и његов идентитет, било то биометријском провером или уносом PIN-а. Друга два бита односе се на прављење резервне копије креденцијала. Подобност за резервну копију (енг. *Backup Eligibility*, BE) показује да креденцијал сме бити сачуван изван аутентификатора који га је креирао, а стање резервне копије (енг. *Backup State*, BS) да ли у том тренутку заиста и јесте. Последња два односе се на садржај самог `authData` записа, индикатор АТ (*Attested Credential Data*) сигнализира присуство атестационог поља, а ED (*Extension Data*) бит указује на присуство резултата проширења. Распоред битова у овом бајту приказан је на слици 5.

flags							
0	1	2	3	4	5	6	7
UP	резервисано	UV	BE	BS	резервисано	AT	ED

Слика 5: Распоред индикатора у бајту заставица.

Јавни кључ кренденцијала представљен је у COSE (*CBOR Object Signing and Encryption*) кључ формату [27]. Овај формат заснива се на бинарном формату CBOR (*Concise Binary Object Representation*) [28]. CBOR формат се заснива на JSON моделу података, али је за разлику од њега компактнији и не захтева обраду текста. Ово га чини погодним за уређаје са ограниченим ресурсима као што су поједини аутентификатори. Запис у CBOR формату је самоописујући, што значи да му није потребна пропратна шема, јер почетни бајт сваке вредности одређује њен тип и податак о њеној дужини. Ово резултује тиме да се крај CBOR записа одређује његовим читањем. Због тога јавни кључ, за разлику од идентификатора кренденцијала, нема испред себе поље са дужином. Запис се додатно скраћује тиме што поља немају текстуална имена, већ их COSE означава малим целим бројевима. Ознака типа кључа је тако број 1, а ознака алгорита број 3. Изглед записа приказан је у листингу 3.

```
{
  1: 2,           // kty, тип кључа, 2 означава EC2
  3: -7,          // alg, алгоритам, -7 означава ES256
 -1: 1,           // crv, крива, 1 означава P-256
 -2: h'9d4f2c...', // x, прва координата тачке, 32 бајта
 -3: h'7a10be...', // y, друга координата тачке, 32 бајта
}
```

Листинг 3: Пример COSE записа кључа, приказан у читљивом облику.

Допуштене вредности поља `alg` прописује регистар који одржава IANA (*Internet Assigned Numbers Authority*) [29]. Првобитно ознаке алгорита нису садржале податак о кривој, већ се она одређивала на основу самог кључа. Због тога су накнадно уведене и ознаке које изричито одређују криву [30]. Облик самог потписа, који се приликом пријаве налази у пољу `signature`, зависи од употребљеног алгорита. Код алгоритама заснованих на ECDSA потпис чине две целобројне вредности, записане правилима DER (*Distinguished Encoding Rules*) нотације [5]. Ознаке које прихвата систем у овом раду дате су у табели 1.

Табела 1: Идентификатори алгоритама у COSE запису кључа.

Ознака	Идентификатор	Алгоритам
ES256	-7	ECDSA са SHA-256
ESP256	-9	ECDSA над кривом P-256 са SHA-256
ES384	-35	ECDSA са SHA-384
ESP384	-51	ECDSA над кривом P-384 са SHA-384
ES512	-36	ECDSA са SHA-512
ESP512	-52	ECDSA над кривом P-521 са SHA-512
EdDSA	-8	EdDSA, крива одређена у кључу
Ed25519	-19	EdDSA над кривом Ed25519
RS256	-257	RSASSA-PKCS1-v1_5 са SHA-256
PS256	-37	RSASSA-PSS са SHA-256

Објекат атестације враћа се једино приликом регистрације и представља CBOR мапу са три поља [5]. Поље `fmt` одређује формат изјаве о атестацији, поље `attStmt` садржи саму изјаву, а поље `authData` је већ описани запис аутентификатора, смештен као низ бајтова. Вредности поља `fmt` и садржај поља `attStmt` описани су у одељку 2.5. Изглед атестационог објекта приказан је у листингу 4.

```
{
  "fmt":      "packed",    // назив формата изјаве о атестацији
  "attStmt":  { ... },     // изјава, чија поља зависе од формата
  "authData": h'...'      // запис аутентификатора, низ бајтова
}
```

Листинг 4: Пример објекта атестације, приказан у читљивом облику.

2.4 Резидентни и нерезидентни приступни кључеви

Према томе да ли их аутентификатор чува код себе, приступни кључеви деле се на нерезидентне и резидентне. Код нерезидентног кључа аутентификатор у својој трајној меморији не чува пар приватног и јавног кључа. Уместо тога овај пар се сваки пут по потреби генерише из семена. Приликом регистрације ово семе се насумично бира. Након одабира, семе се у шифрованом облику шаље сервису као идентификатор креденцијала. Заједно с њим шаље се и генерисани јавни кључ, док се приватни кључ одбацује. Касније, када сервис жели да изврши аутентификацију корисника, неопходно је да аутентификатору пошаље шифровано семе назад да би потписивање

изазова било могуће. Ово захтева од сервиса да унапред зна који корисник покушава да се пријави, пре него што захтев може да се пошаље аутентификатору [1].

Код резидентног кључа (енг. *client-side discoverable credential*) аутентификатор пар приватног и јавног кључа чува код себе, заједно са идентификатором сервиса и корисничког налога. На овај начин аутентификатор је у стању да пронађе и понуди креденцијал за пријаву на сервис чак и када му се не пошаље идентификатор креденцијала. Међутим, такви кључеви троше ограничену меморију аутентификатора [1].

Само поседовање аутентификатора ипак није довољно, јер би тиме изгубљен или украден уређај омогућио приступ налогу, а при томе би чак могао и сам да понуди о ком налогу је реч [1]. Из тог разлога се уз резидентни кључ захтева и верификација корисника, најчешће уз помоћ PIN-а или биометрије. Пријава овим добија и други фактор аутентификације. За разлику од лозинке, PIN се не шаље преко мреже и не може процурети у крађи базе података, а број покушаја ограничава сам уређај [1].

2.5 Атестација

Атестација је потписана тврдња о произвођачу и моделу аутентификатора, као и о томе да је приватни кључ настао у њему [5]. Сервису она служи да утврди где се приватни кључ креирао и где се налази. То може бити поуздано извршно окружење (енг. *Trusted Execution Environment*, TEE), хардверски издвојено на главном процесору, или сигурни елемент (енг. *secure element*) са сопственим процесором и меморијом. Ова тврдња потписује се посебним паром кључева за атестацију уграђеним у уређај приликом производње.

Потпис покрива запис аутентификатора и сажетак података клијента, те атестација важи само за захтев поводом ког је издата [5]. Сама изјава налази се у пољу `attStmt`, док поље `fmt` одређује ком формату изјава припада. Код формата заснованих на ланцу сертификата, поље `attStmt` садржи ознаку алгорита `alg`, потпис `sig` и ланац сертификата `x5c` [5]. Први сертификат у ланцу садржи јавни кључ за атестацију, којим се потпис проверава. Остатак ланца се састоји од посредних сертификата и завршава се коренским сертификатом произвођача. Овај коренски сертификат је потребно да сервис познаје унапред.

Подразумевана вредност поља `attestation` је *none*, те сервис који жели проверу атестације мора је изричито затражити. Вредност *direct* значи да сервис тражи изјаву онакву какву је аутентификатор формирао, док *indirect* препушта клијентској апликацији да је пружи на начин који сматра прикладним [5]. Уколико сервис ништа не затражи, клијентска апликација поље `fmt` поставља на *none*, а поље `attStmt` на празну мапу, чиме изјава нестаје иако ју је аутентификатор формирао [5]. Разлог за овакво подразумевано понашање је приватност. Сертификат својствен једном уређају потенцијално би омогућио повезивање корисничких налога на различитим сервисима

ако за пријаву користе исти аутентификатор. Из тог разлога произвођачи један пар кључева за атестацију уграђују у целу серију уређаја, уз препоруку да исти сертификат дели најмање сто хиљада уређаја [5].

2.6 Проширења намењена за енкрипцију

Проширења су додатне могућности које сервис може затражити током захтева. Проширење PRF (*Pseudo-Random Function*) омогућава коришћење приступног кључа као извора тајни за симетричну енкрипцију. Током креирања креденцијала аутентификатор поред пара приватног и јавног кључа такође бира и памти две насумичне вредности од величине 32 бајта [6].

Током церемоније пријаве платформа шаље со, на основу које аутентификатор рачуна HMAC-SHA-256 вредност. Со има улогу поруке, а придружена тајна улогу кључа [6]. Добијена вредност је увек иста за исти креденцијал и исту со, а враћа се платформи назад као резултат проширења. Различите соли резултују у међусобно неповезаним вредностима. С обзиром да придружена тајна не напушта уређај, вредност се не може израчунати изван њега.

Начин на који со настаје зависи од тога ко позива аутентификатор. Позив преко СТАР-а прима со непосредно од позиваоца, као низ дужине 32 бајта [6]. За разлику од тога, прегледач креира со из дате улазне вредности по одређеном правилу [5]:

$$so = \text{SHA-256}(\text{"WebAuthn PRF"} \parallel 0x00 \parallel \text{улаз})$$

Хеширање је потребно зато што со мора имати тачно 32 бајта, док улазна вредност може бити произвољне дужине [5][6]. Основни облик проширења прима со само у церемонији пријаве, па сервис коме је кључ за енкрипцију потребан већ приликом регистрације мора спровести и пријаву [1]. СТАР 2.2 уводи облик који со прима и при регистрацији, па је за цео процес довољна и једна церемонија [6].

2.7 Хибридни транспорт

Хибридни транспорт омогућава да се аутентификатор налази на другом уређају, најчешће телефону, док се пријава спроводи на рачунару [6]. Рачунар приказује QR (*Quick Response*) код који телефон скенира камером, како би се помоћу Bluetooth везе огласио рачунару. Пријем тог оглашавања доказује физичку близину уређаја. Без тог доказа било би довољно да нападач на сопственој страници прикаже QR код и наведе корисника да га скенира [6].

У оглашавању телефон наводи домен тунела и податке о његовој путањи [6]. Рачунар и телефон повезују се на тај тунел путем WebSocket везе. Кроз тунел затим теку СТАР поруке, шифроване кључевима изведеним из QR кода, тако да услуга која пружа тунел не може прочитати њихов садржај [6].

Глава 3

Архитектура система

У овом поглављу приказана је архитектура система. Прво су наведени захтеви које систем испуњава, затим улога сваког од три програма који чине систем и њихова међусобна веза. Потом се налази модел претњи и ограничења система, а на крају коришћене технологије.

3.1 Циљеви и захтеви система

Полазна замисао система јесте веб апликација за чување белешки и датотека са системом аутентификације базираним на приступним кључевима. Циљ рада је да покрије цео пут тог кључа, од настанка у аутентификатору до његове провере и употребе на серверу. Како би се лозинке потпуно избегле у имплементацији система, враћање приступа налогу у случају губитка приступног кључа реализује се помоћу магичних веза (енг. *magic links*) које се шаљу е-поштом.

Други принцип тиче се онога што бива уписано на диск сервера. Подаци који могу идентификовати корисника, као и његове белешке и датотеке, на диску се чувају искључиво у шифрованом облику. Како би овај циљ имао смисла, кључеви за њихово шифровање се такође никада не складиште у отвореном облику. Исти принцип примењен је и на тајне вредности које сервер користи приликом покретања. Све ове тајне шифроване су, а отварају се искључиво помоћу приступних кључева. Њихов отворен облик налази се искључиво у радној меморији (енг. *Random Access Memory, RAM*).

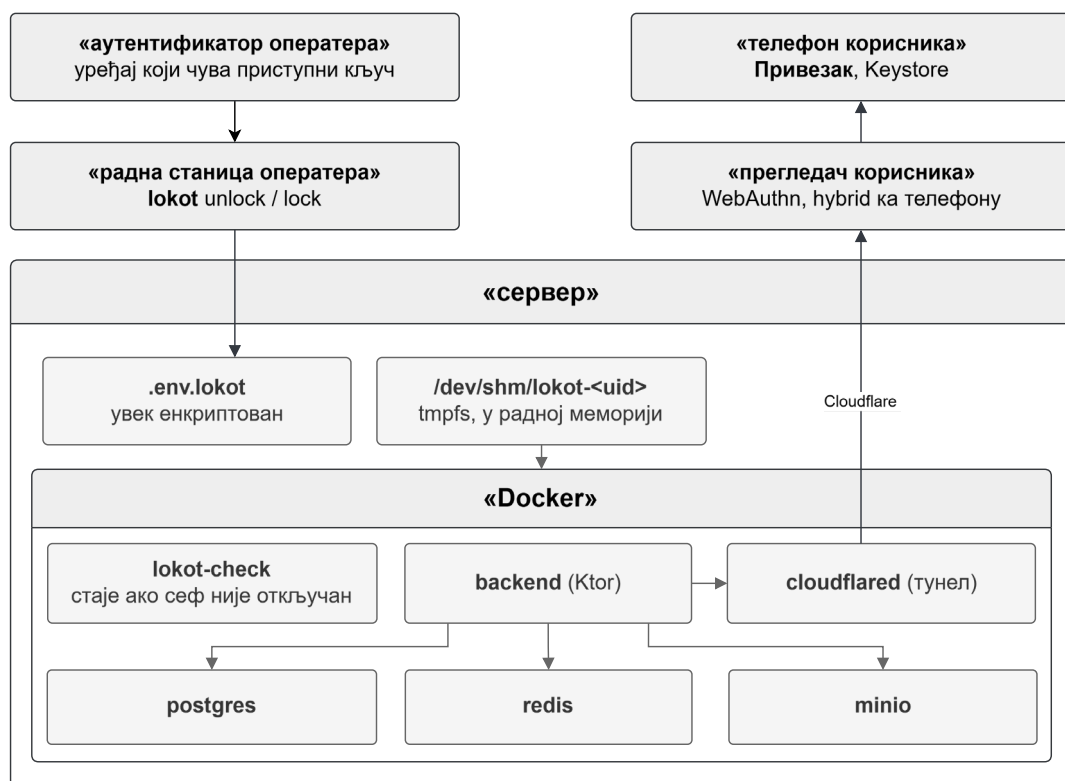
Трећи циљ рада је демонстрирање атестације. У раду се она огледа тако што је приступ датотекама резервисан једино за кључеве настале уз помоћ аутентификатора описаног у овом раду. Из ових циљева произлазе следећи функционални захтеви:

- пријава на веб апликацију приступним кључем, без лозинке,
- додела новог приступног кључа уз помоћ магичне везе послате е-поштом,
- стварање приступних кључева у безбедном хардверу телефона,
- прилагање потписане тврдње о уређају уз сваки створени кључ,
- провера те тврдње на серверу и додела права на основу ње,
- чување белешки и датотека, при чему датотеке захтевају проверену тврдњу,
- извођење кључева за симетричну енкрипцију из истог приступног кључа и
- чување тајни сервера у сефу који се отвара приступним кључем.

3.2 Три подсистема и веза међу њима

Привезак ствара кључ и сведочи о уређају у ком је настао. Креденац ту тврдњу проверава и на основу ње одређује ниво приступа корисника. Локот користи креденцијал у сврху извођења кључа за шифровање, тако да тајне без којих се Креденац не може покренути чува у сефу. Поред конзолне апликације, Локот постоји и као библиотека помоћу које Креденац може да приступи сефу.

Креденац се извршава на машини која је изложена интернету уз помоћ *Cloudflare* тунела [31]. Компоненте овог система се покрећу у засебним *Docker* контејнерима [32]: позадински део (енг. *backend*), база података, привремено складиште записа и складиште датотека. Позадински део опслужује два домена. Први служи саму апликацију, а други служи страницу за откључавање серверских тајни док је сеф закључан. Подизање система почиње од провере стања сефа. Ако је сеф закључан, остали контејнери се не покрећу. Откључане тајне не уписују се на диск, већ у датотеке привременог система датотека (енг. *temporary file system*, *tmpfs*) који се налази у радној меморији. Распоред је приказан на слици 6.



Слика 6: Распоред компоненти система.

3.3 Модел претњи и ограничења система

Модел претњи полази од претпоставке да се Креденац извршава на једној машини којом управља један оператер. Веза међу контејнерима је шифрована, што штити од прислушкивања на самој машини. Претње које систем обухвата и мере које на њих одговарају приказане су у табели 2.

Табела 2: Претње и мере заштите.

Претња	Мера заштите
копија базе података или складишта датотека	подаци су шифровани, а кључеви се не налазе у бази
право уписа у базу података	измена записа открива се провером контролне ознаке
други кориснички налог на истој машини	тајне се налазе у радној меморији у директоријуму доступном само корисничком налогу оператера
клонирани аутентификатор	понављање открива бројач употребе кључа
копија складишта апликације на телефону	кључ за податке умотан је кључем из безбедног хардвера, тако да погађање PIN-а мора бити на самом телефону
уобичајени напади на веб апликације	посебан токен уз сваки захтев, строга политика садржаја, забрана уграђивања у туђу страницу и ограничење броја захтева

Изван домена заштите остају администраторски приступ машини док систем ради, компромитован прегледач или телефон и компромитовани пружалац тунела који прекида TLS (*Transport Layer Security*) [33][34] пре него што саобраћај стигне до машине. Губитак свих уписаних кључева за Локот је ненадокнадив, што је последица тога да тајне не постоје нигде изван сефа и аутентификатора.

3.4 Коришћене технологије

Сва три подсистема написана су у језику *Kotlin*. Привезак је *Android* апликација са корисничким интерфејсом у *Jetpack Compose* развојном оквиру. Позадински део Креденца користи *Ktor*, док је интернет страница писана у *TypeScript*-у, без посебног развојног оквира. Локот је написан у *Kotlin Multiplatform* оквиру, тако да се преводи и у изворни (енг. *native*) програм и у *JVM (Java Virtual Machine)* библиотеку. Подаци се чувају у *PostgreSQL* бази, а записи са кратким роком трајања у *Redis* бази. Датотеке се складиште у *MinIO* систему за складиштење објеката. За рад са форматима које прописују *WebAuthn* и *CTAP* нису коришћене специјализоване библиотеке, будући да је управо рад са њима предмет изучавања.

Глава 4

Привезак

У овом поглављу описан је Привезак. Прво је наведено шта он јесте, а шта намерно није, потом начин на који се уклапа у *Android* системски оквир за креденцијале. Потом следе хијерархија кључева коју користи и њихова заштита PIN-ом. Затим описује формирање аутентификаторског одговора, атестација кључа из безбедног хардвера телефона, као и одговор на захтеве PRF проширења. На крају су описана додатна очвршћавања, провера решења и његова ограничења.

4.1 Улога подсистема и намерна ограничења

Привезак је добављач креденцијала (енг. *credential provider*) за *Android* систем. Када страница или апликација на телефону затражи приступни кључ, систем приказује списак свих добављача и кључеве које они нуде, што укључује и Привезак. Привезак ствара искључиво резидентне приступне кључеве и увек спроводи верификацију корисника.

Аутентификатор све податке чува локално, без креирања налога, коришћења сервера или синхронизације. Због тога апликацији није потребна ниједна мрежна дозвола за исправно функционисање. Једина дозвола коју апликација тражи јесте приступ камери, али она није обавезна. Ова дозвола омогућава кориснику да скенира QR код и аутентификује се на другом уређају.

Апликација нема начин да утврди да ли је у систему изабрана као добављач креденцијала. Документација предвиђа само отварање системских подешавања, али не и проверу њиховог стања [35]. Због тога се приликом првог покретања апликације приказује једнократни дијалог који корисника усмерава на одговарајућу страницу системских подешавања. Привезак не може да утврди избор корисника, већ се ослања на то да ће корисник самостално извршити потребно подешавање.

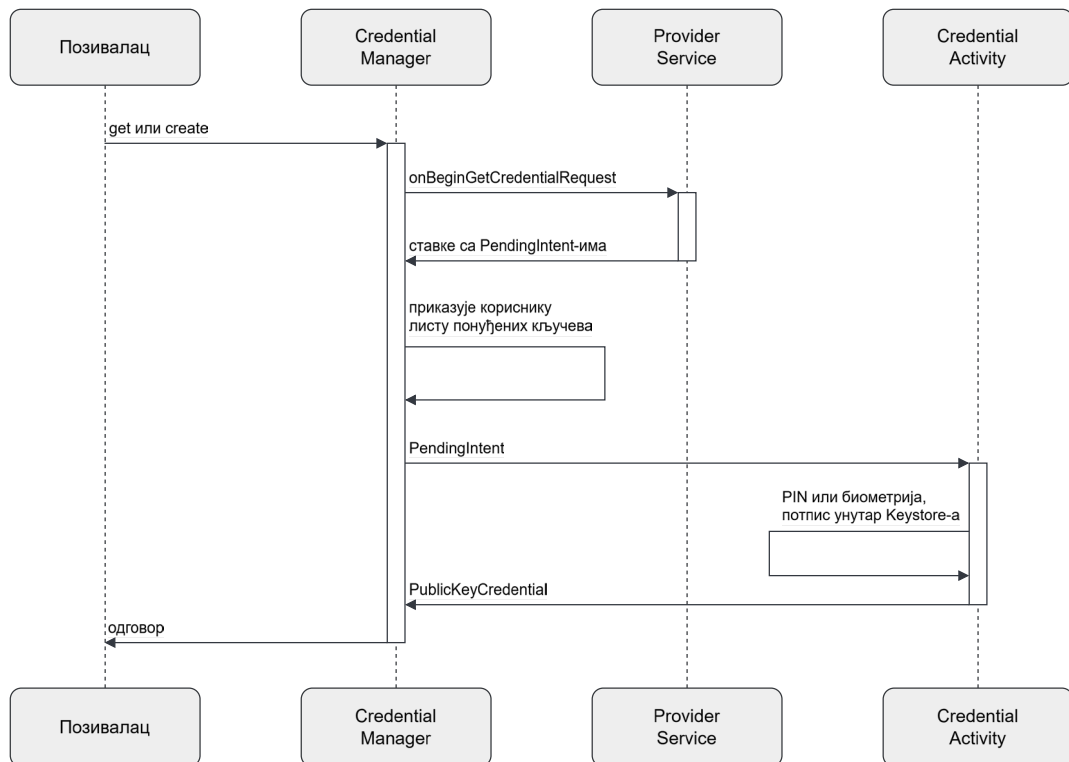
4.2 Интеграција са системским посредником *Credential Manager*

Модел добављача креденцијала у *Android*-у обликован је кроз два ограничења. Та ограничења су да приликом састављања списка кључева систем позива добављача без корисничког интерфејса и од њега очекује да одговори брзо. Због тога добављач може једино да врати податке. Све што тражи интеракцију са корисником врши се

касније преко одложене намере (енг. *Pending Intent*) приложене уз изабрани кренденцијал. Да би систем уопште препознао Привезак као добављача кренденцијала, то је неопходно назначити у манифесту апликације. Уз то иду посебна дозвола и опис онога што добављач нуди, у овом случају само приступне кључеве, без лозинки [35].

Шта ће добављач вратити зависи од тога да ли је Привезак откључан. Ако није, корисник на листи види једино могућност откључавања. Ако јесте, добављач међу сачуваним кренденцијалима бира и шаље назад систему оне који одговарају идентификатору сервиса и листи дозвољених кренденцијала коју шаље сервис. Приликом захтева за регистрацију, нуди се само креирање новог кључа.

Када корисник изабере једну од понуђених ставки, систем покреће приложену намеру и отвара екран Привеска, који тада тражи PIN или биометријску пријаву. Након потврде идентитета корисника, екран саставља одговор на основу тога шта му је било прослеђено. Уз захтев за пријаву враћа се потписани одговор затраженог кренденцијала, а уз захтев за регистрацију новонастали јавни кључ. Ако је корисник изабрао само откључавање, уместо одговора о кренденцијалу, апликација поново саставља листу и враћа је систему. Цео ток приказан је на слици 7.

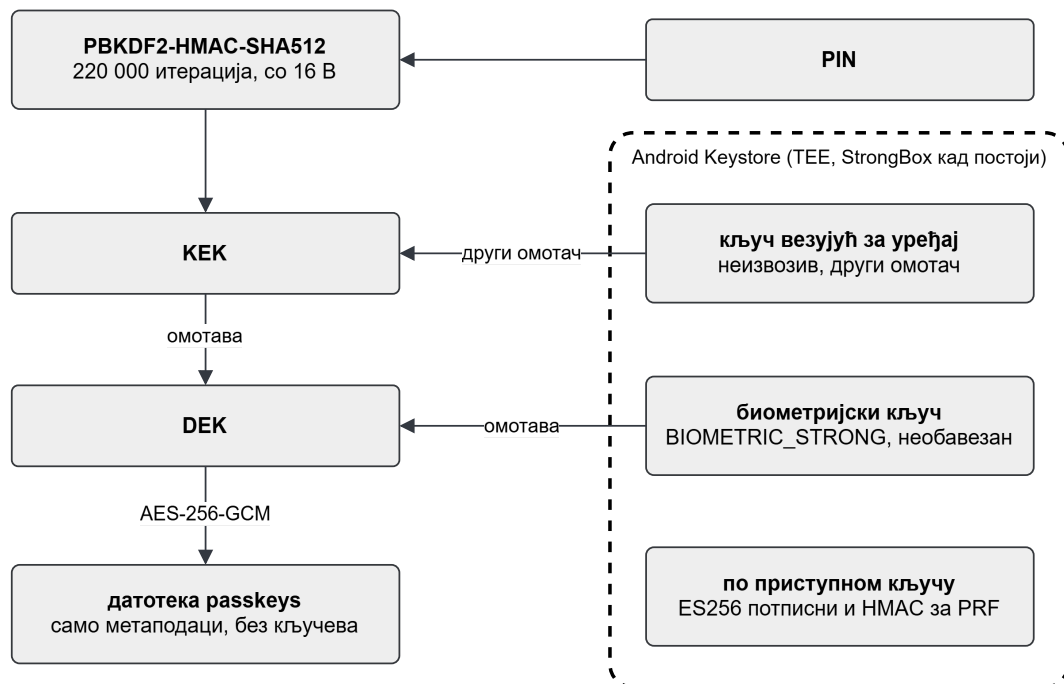


Слика 7: Ток обраде захтева у Привеску.

4.3 Хијерархија кључева, заштита PIN-ом и чување метаподатака

Приватни кључеви никада не напуштају *Android Keystore*. Реч је о системском делу који кључеве чува или у TEE или у сигурном елементу као што је StrongBox [36]. *Keystore* не дозвољава апликацијама да заправо виде кључеве које креирају, већ искључиво да их користе кроз унапред дефинисани интерфејс. Овај интерфејс такође гарантује да једина апликација која може користити кључеве јесте она која их је и креирала. Привезак зато не чува кључеве, већ само метаподатке о њима, у шта спадају идентификатор креденцијала, назив и идентификатор сервиса за који је намењен, име и идентификатор корисника, бројач употребе и време настанка.

Ови метаподаци налазе се у датотеци у приватном простору апликације, шифровани кључем за податке (енг. *Data Encryption Key*, DEK). Сам DEK такође није сачуван у отвореном облику, већ се уместо тога шифрује другим кључем. Ово шифровање се назива умотавање, а други кључ се назива кључем за енкрипцију кључа (енг. *Key Encryption Key*, KEK). KEK се изводи из PIN-а употребом PBKDF2 функције са 220.000 итерација и насумичном соли дужине 16 бајтова [16]. Тако умотан DEK умотава се још једном кључем из *Keystore*-а. Први ниво умотавања успорава сваки појединачни покушај, док други ниво спречава испробавање PIN-а ван самог телефона, чак и у случају да нападач дође у посед умотаног кључа.



Слика 8: Хијерархија кључева у Привеску.

Откључавање биометријом је опционо и служи првенствено унапређењу корисничког искуства (енг. *User Experience*, UX). У случају да корисник одлучи да га користи, за њега се ствара посебан КЕК у *Keystore*-у. Његова употреба захтева биометријску проверу и престаје да важи када се биометријски подаци на уређају промене. Тај кључ умотава исти ДЕК, па оба начина откључавања воде до исте тајне.

Сваком приступном кључу који Привезак креира припадају два кључа у *Keystore*-у, један за потписивање и један за PRF проширење. Оба кључа се прво покушавају креирати у *StrongBox*-у. Ако га телефон не поседује, кључеви се стварају и чувају у TEE. Брисањем приступног кључа из Привеска бришу се и обе вредности. Хијерархија кључева приказана је на слици 8.

Након откључавања ДЕК остаје у меморији одређено време, подразумевано петнаест минута, а то раздобље назива се сесијом. Трајање сесије може да подешава корисник, од моменталног истицања до неограниченог трајања. Без сесије корисник би морао да се верификује два пута, једном да би се саставила листа креденцијала, а затим поново да би кључ за изабрани креденцијал био употребљен. Листу саставља добављач, а за то су му неопходни метаподаци који се отварају са ДЕК-ом. Док сесија траје добављачу је омогућено креирање листе креденцијала без потребе да корисника пита за PIN. Међутим, валидна сесија није довољна како би се одабран креденцијал употребио за потписивање, већ та радња захтева поновну верификацију корисника.

Број PBKDF2 итерација није произвољан, већ су вредности узете из OWASP PBKDF2 препоруке [37]. За SHA-256 она износи 600.000 итерација а за SHA-512 она је 220.000 итерација. Одлука о коначном избору донета је на основу резултата мерења приказаних у табели 3. Како су обе комбинације алгоритма и итерација препоручене, изабрана је она која на тестираним уређајима захтева мање времена, а то је SHA-512 са 220.000 итерација. Мерење је спроведено на два уређаја различитих генерација, са десет наизменичних извођења, укљученим екраном и без других оптерећења.

Табела 3: Измерена трајања извођења кључа помоћу PBKDF2.

Уређај	Комбинација	Најмање (ms)	Просек (ms)	Највише (ms)
Nubia Z60 Ultra Snapdragon 8 Gen 3 (2023)	SHA-256 600.000	2343	2352,5	2364
	SHA-512 220.000	994	1003,6	1009
OnePlus 7T Pro Snapdragon 855+ (2019)	SHA-256 600.000	3317	3327,2	3335
	SHA-512 220.000	1572	1579,2	1585

4.4 Формирање одговора аутентификатора

Привезак одговор аутентификатора саставља без коришћења специјализоване библиотеке за CBOR формат. Уместо тога подржани су CBOR типови које одговор треба да садржи: мапе, низови, низови бајтова, текст и цели бројеви. Уз помоћ њих се формирају COSE кључ и објекат атестације. Запис аутентификатора није CBOR мапа, већ низ поља утврђеног распореда, на чијем крају се налази COSE кључ [5].

У запису аутентификатора Привезак поставља индикаторе присуства и верификације корисника, а приликом регистрације и индикатор атестираног креденцијала. Кључеви су намењени искључиво за коришћење уз ES256 алгоритам са P-256 кривом. Због тога COSE кључ увек чини иста мапа са пет поља. Привезак објекат атестације саставља са изјавом, осим када изостане ланац сертификата из *Keystore*-а.

Ко саставља податке клијента зависи од позиваоца. Када захтев долази од прегледача, саставља их он, а аутентификатору прослеђује само њихов сажетак [35]. У том случају Привезак потписује запис аутентификатора заједно са тим сажетком, а поље `clientDataJSON` остаје празно. Када захтев долази из апликације на телефону, Привезак ове податке саставља сам. Као порекло тада се уписује облик који *Android* прописује за апликације, изведен из сажетка сертификата којим је апликација потписана [35]. Име пакета позиваоца налази се у засебном пољу `androidPackageName` [38].

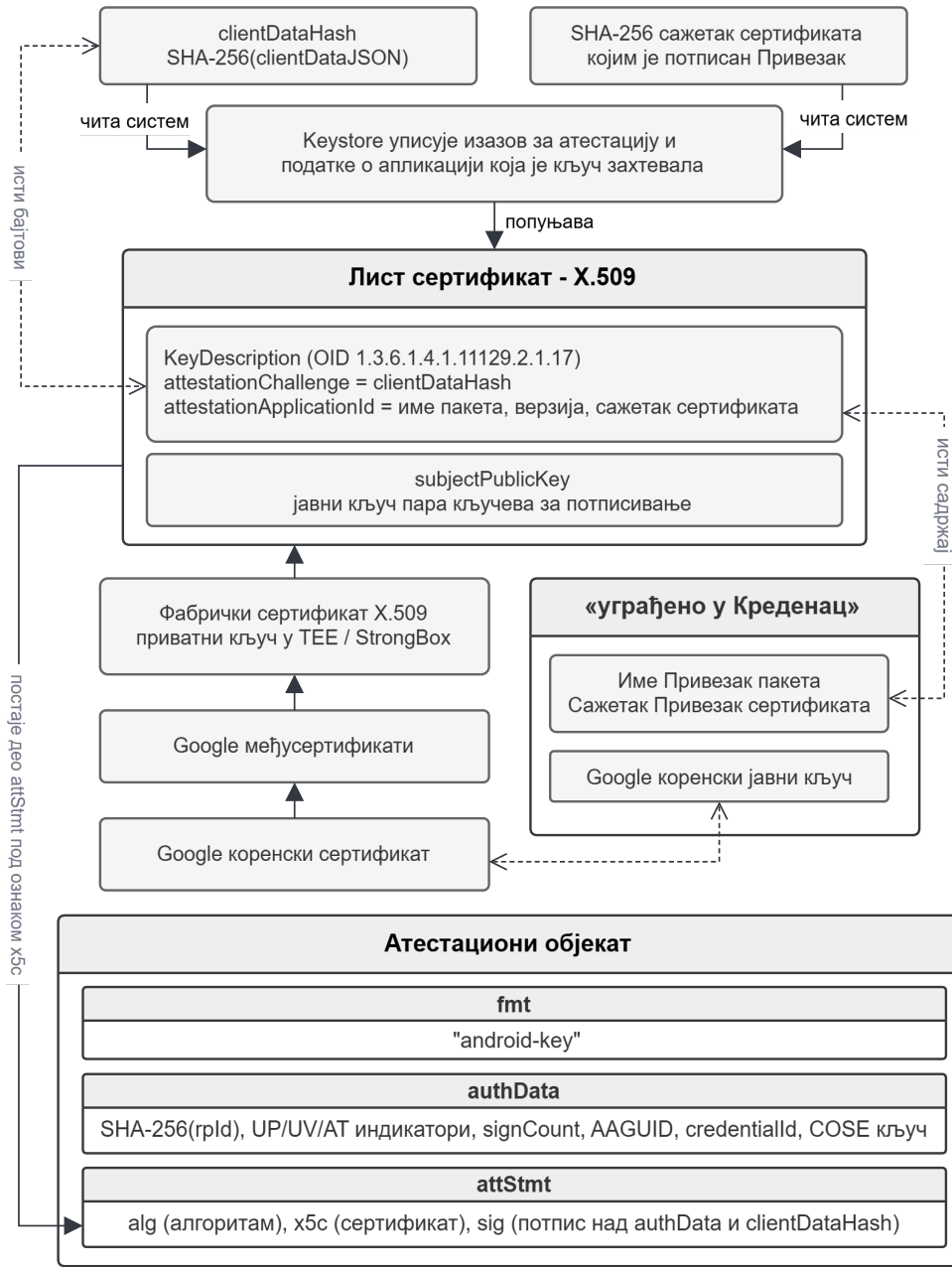
4.5 Атестација кључа из безбедног хардвера

Када Привезак у *Keystore*-у креира пар кључева за потписивање, њима се такође придружује и ланац сертификата. Први сертификат у том ланцу садржи јавни кључ тог пара, као и податке о атестацији [39]. Међу тим подацима се налази и изазов који је сажетак података клијента те церемоније регистрације. На овај начин ланац важи само за регистрацију приликом које је настао.

Изјаву о атестацији Привезак саставља у *android-key* формату. Поље `alg` садржи ознаку ES256, а поље `x5c` ланац сертификата. Потпис у пољу `sig` покрива запис аутентификатора и сажетак података клијента. Уместо посебног кључа за атестацију, Привезак потпис ствара приватним кључем самог креденцијала [5].

Подаци о атестацији чине `KeyDescription` структуру уписану у сертификат као проширење под ознаком 1.3.6.1.4.1.11129.2.1.17 [40]. Поред изазова, у њој се налазе и ниво заштите са којим се кључ чува, као и поље `attestationApplicationId`. Ово поље именује апликацију која је креирала кључ, на основу имена пакета и сажетка сертификата којим је апликација потписана [40]. Овај ланац завршава се са неким од *Google*-ових коренских сертификата за хардверску атестацију [39].

Сервис провером ланца утврђује две ствари. Прва је да се приватни кључ налази у безбедном хардверу телефона. Друга је да га је створио Привезак, на шта указује поље attestationApplicationId. Процес ове провере описан је у одељку 5.4. Цео пут, од сажетка података клијента до објекта атестације, приказан је на слици 9.



Слика 9: Настанак атестације и ланац сертификата.

4.6 Одговор на PRF захтеве

Уз сваки приступни кључ Привезак чува и тајну намењену PRF проширењу. Од улазне вредности коју сервис пошаље, креира се со на начин описан у одељку 2.6. Привезак затим над сољу рачуна HMAC, користећи тајну као кључ.

Улазне вредности сервис наводи у једном од два поља. Поље `eval` садржи улазне вредности које важе без обзира на то који је креденцијал изабран. За разлику од њега, поље `evalByCredential` наводи посебне вредности за сваки идентификатор креденцијала, па онда на основу тога Привезак користи одговарајуће вредности.

Захтев не стиже увек из прегледача на самом телефону. Приликом пријаве са рачунара телефон служи као аутентификатор преко хибридне везе. Со тада настаје на рачунару и до телефона стиже већ израчуната. *Android* у том случају проширење прослеђује под називом `prfAlreadyHashed` [1]. Привезак ту вредност користи у оригиналном облику, без додавања префикса и без хеширања. Ако би је хеширао поново, исти улаз би резултовао другачијом вредношћу у зависности од тога да ли захтев долази директно са телефона или са другог уређаја.

Привезак на PRF захтев одговара у обе церемоније. Уз регистрацију одговор садржи и ознаку `enabled`, којом се потврђује да је проширење за тај кључ доступно. Поред ње стижу и израчунате вредности, тако да сервис добија кључ за шифровање већ током регистрације. Ова надоградња уведена је уз СТАР 2.2, као што је то и описано у одељку 2.6. Захтев и одговор приказани су у листингу 5.

```
"extensions": {                                     // проширење у захтеву сервиса
  "prf": {
    "eval": {
      "first": "0KHQotCj0JTQ",                       // улазна вредност у base64url запису
      "second": "ldCd0KLQmCD"                       // опциона друга улазна вредност
    }
  }
}

"clientExtensionResults": {                         // проширење у одговору Привеска
  "prf": {
    "enabled": true,                                 // само уз регистрацију
    "results": {
      "first": "Qn9Ce0JHQLd...",                   // излаз HMAC функције, 32 бајта
      "second": "CC0KPQiNCj..."
    }
  }
}
```

Листинг 5: Пример PRF захтева и одговора.

4.7 Апликација и додатна очвршћавања

Највећи део посла Привезак обавља као добављач, док сама апликација служи прегледу приступних кључева и подешавањима поменутим у одељку 4.3. Поред тога апликација садржи и читач QR кода који једино прихвата кодове са префиксом FIDO:/. Прочитану везу апликација предаје систему, који даље спроводи хибридни транспорт са другим уређајем. Привезак у тој комуникацији не учествује, те му мрежна дозвола није потребна. Привезак се укључује тек када се затражи одговор аутентификатора.

Оба екрана апликације постављају заставицу FLAG_SECURE. Систем тиме онемогућава снимање садржаја екрана, било као слике или видео записа. У списку недавно коришћених апликација приказује се празан прозор. *Android* допушта да поља за унос попуњавају друге апликације, које због тога могу видети шта се на екрану налази. Привезак ту могућност на својим екранима искључује, тако да се поље за PIN таквим апликацијама не приказује. Одласком у позадину, апликација се закључава, осим приликом промене конфигурације као што је ротирање екрана. Сесија описана у одељку 4.3 при томе тече независно, јер припада добављачу, а не екрану апликације. Систем такође нема дозволу да прави резервну копију података апликације.

4.8 Тестирање и ограничења

Сви тестови Привеска су инструментални (енг. *instrumented tests*), јер *Keystore* постоји једино на правом уређају. Тестови покривају шифровање метаподатака, потписивање, извођење вредности за PRF и атестацију над задатим изазовом. Такође се проверава и да ли кључеви заиста настају у безбедном хардверу.

Поверење у атестацију своди се на поверење у фабричке кључеве којима је ланац потписан. У пракси дешава се да такви кључеви процуре, због чега *Google* објављује списак повучених сертификата [39]. Уз сваку ставку наведен је и разлог, на пример компромитовани кључ. Провера тог списка описана је у одељку 5.4.

Постоје и модули попут *TrickyStore*-а, који на нивоу *Keystore*-а подмећу процурели кључ за атестацију, па у подацима о атестацији нема знакова да је уређај измењен [41]. У литератури је описан и поступак у ком се стварање кључа може пренети и на неизмењени уређај, тако да се добијени ланац може користити на модификованом уређају [41]. Ланац тада јесте валидан, али сведочи о другом уређају, а не о оном који заправо тај ланац користи. Ниједна од ове две слабости не потиче од Привеска, већ од принципа на ком поступак атестације почива. Процена поузданости самог уређаја зато остаје засебна тема, коју рад не обухвата.

Глава 5

Креденац

У овом поглављу описан је Креденац. Прво је приказана његова архитектура, програмски интерфејс и модел података, а затим регистрација помоћу магичних веза. Потом следе провере које сервер спроводи над церемонијама пријаве и регистрације и начин на који порекло приступног кључа утиче на приступна права корисника. Након тога описани су сесије и заштита веб слоја, подаци у мировању, те отпремање и преузимање датотека. На крају се налазе *Signal* интерфејс, тестирање и ограничења.

5.1 Архитектура, API и модел података

Креденац се дели на страницу коју корисник отвара у прегледачу и њен позадински део који се извршава на серверу. Позадински део служи програмски интерфејс (енг. *Application Programming Interface*, API). Адресе за приступ њему почињу са /api префиксом и покривају аутентификацију, белешке и датотеке. Уз то позадински део испоручује и саму страницу, чије су датотеке унапред изграђене. Страница и интерфејс тиме деле исто порекло. Све путање приказане су у табели 4.

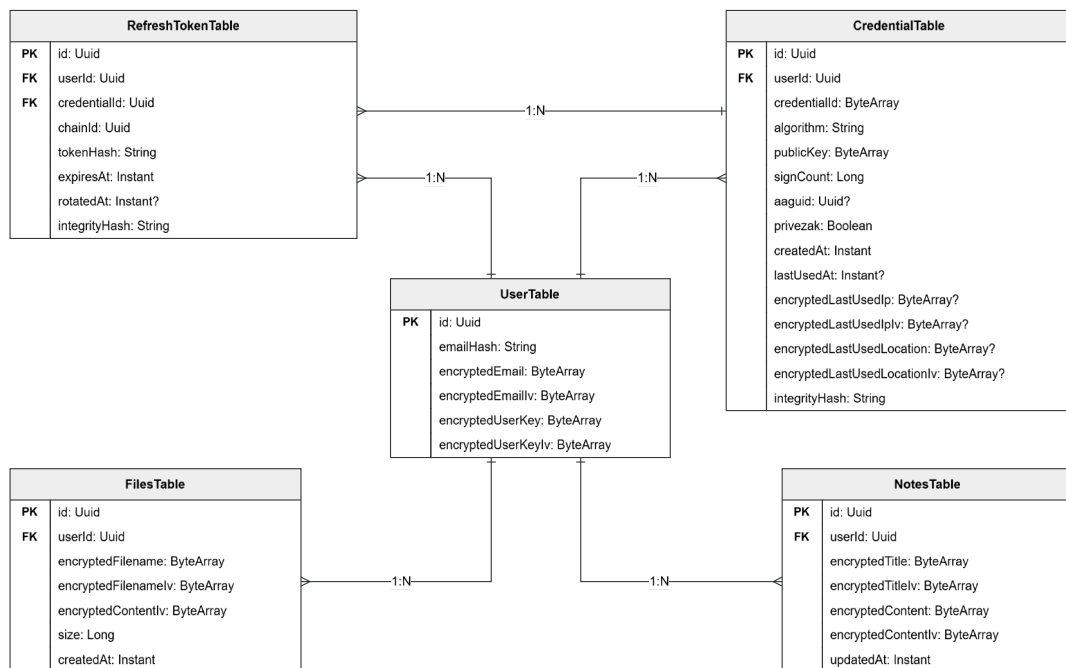
Табела 4: Путање програмског интерфејса.

Метода	Путања	Опис
POST	/auth/register/verify	слање магичне везе на адресу е-поште
POST	/auth/register/start	издавање изазова за регистрацију
POST	/auth/register/finish	провера одговора и упис новог кључа
POST	/auth/login/start	издавање изазова за пријаву
POST	/auth/login/finish	провера потписа и отварање сесије
POST	/auth/refresh	обнова сесије новим токеном
POST	/auth/logout	поништавање ланца токена за освежавање
GET	/auth/credentials	списак приступних кључева налога
DELETE	/auth/credentials/{id}	уклањање приступног кључа
POST	/auth/credentials/add	додавање новог кључа уз потврду постојећим
POST	/auth/reauth	поновна потврда идентитета
DELETE	/auth/account	брисање налога
GET, POST	/notes	списак белешки и стварање нове
PUT, DELETE	/notes/{id}	измена и брисање белешке
GET, POST	/files	списак датотека и отпремање нове
GET, DELETE	/files/{id}	преузимање и брисање датотеке

Улоге унутар позадинског дела подељене су у три слоја. Први слој прима захтев и проверава његов облик, други спроводи логику апликације, а трећи приступа бази и складиштима привремених и трајних података. Поред њих, постоји и засебна целина у којој се врши рад са WebAuthn форматима, од провере церемонија до обраде ланца атестације. На тај начин, све одлуке о поверењу доносе се на једном месту.

У бази постоји пет табела података, табела корисника, њихових кренцијала, токена за освежавање, белешки и датотека. Сви подаци који могу идентификовати корисника складиште се искључиво у шифрованом облику. Због тога адресе е-поште у бази претражују се на основу њихових хеш вредности. Садржај датотека корисника не налази се у бази, већ у складишту објеката. Модел је приказан на слици 10.

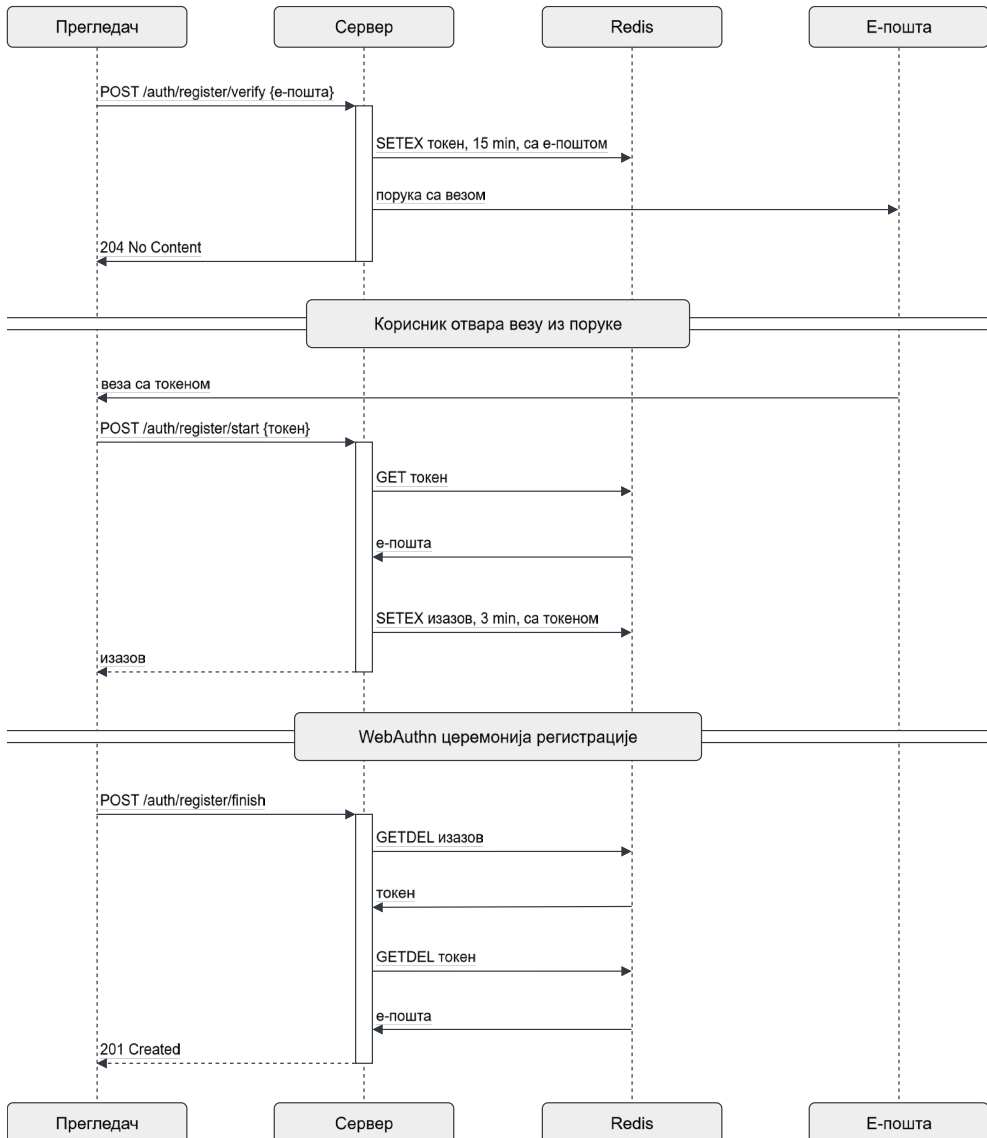
Сваки ред у корисничким табелама припада само једном налогу. Функције које читају те редове и мењају њихов садржај захтевају податак о власнику. Овај податак прослеђује кроз параметар контекста (енг. *context parameter*). Позив без тог податка преводилац (енг. *compiler*) одбија, док сваки прихваћени позив функције подразумева ограничење приступа на одређеног власника. На овај начин се онемогућава приступ туђим подацима у бази података.



Слика 10: Модел података Креденца.

5.2 Регистрација магичном везом

С обзиром на то да Креденац не користи лозинке, у случају губитка приступног кључа систем за опоравак налога (енг. *recovery system*) шаље магичну везу на адресу е-поште корисника. Сервер уписује насумичан токен са трајањем од петнаест минута у *Redis* и потом кориснику шаље везу са тим токеном. Веза послата на непознату адресу отвара нови налог, а за постојећи додаје нови приступни кључ. Сервер у оба случаја одговара исто, чиме се онемогућава откривање адреса са постојећим налогом.



Слика 11: Ток регистрације магичном везом.

Отварањем везе страница започиње церемонију регистрације. Сервер из токена чита адресу, проналази или ствара налог и шаље изазов за потписивање. Уз изазов се шаље и колачић (енг. *cookie*) изведен из њега, тако да церемонију може завршити једино прегледач који ју је започео. Токен и изазов уписују се у *Redis* у виду својих НМАС вредности. Запис изазова при томе упућује на запис токена из е-поште, те се завршетком церемоније оба уклањају из привремене базе. Токени се троше тек након успешне провере, па неуспешни покушаји не поништавају везу. Цео ток приказан је на слици 11.

Е-поштом кориснику стиже порука и када му се кључ дода или уклони, када се открије клонирање аутентификатора или када му се налог обрише. Поруке које се односе на кључеве садрже и податке о догађају као што су коришћени прегледач и уређај, IP адреса, место и време догађаја. На основу овога корисник разликује своје радње од туђих. Неочекивана порука је први знак да неко други има приступ налогу.

5.3 Провера регистрације и пријаве на серверу

Пре сваке церемоније страница од сервера тражи изазов. Сервер уз њега шаље свој идентификатор ослањајуће стране. Током регистрације шаље се такође и кориснички идентификатор, адреса е-поште и списак кључева које налог већ поседује. Аутентификатор тај списак користи како не би креирао дуплиране кључеве за исти налог. Од тих вредности страница формира опције које предаје прегледачу, при чему увек тражи верификацију корисника и атестацију кључа. Структуре са опцијама за регистрацију и пријаву дате су у листинзима 6 и 7.

```
PublicKeyCredentialCreationOptions {
  rp: { id: "kredenac.moma.rs", name: "Kredenac" },
  challenge: h'...', // изазов који даје сервер
  user: {
    id: h'...', // кориснички идентификатор
    name: "ime.prezime@email.com",
    displayName: "ime.prezime@email.com"
  },
  pubKeyCredParams: [ ... ], // ES256, RS256 или EdDSA
  authenticatorSelection: {
    residentKey: "required",
    userVerification: "required"
  },
  attestation: "direct",
  excludeCredentials: [ ... ], // кључеви које налог већ има
  timeout: 60000
}
```

Листинг 6: Опције за церемонију регистрације.

```

PublicKeyCredentialRequestOptions {
  challenge: h'53545544454E544920504F4245C490554A55', // изазов од сервера
  rpId: "kredenac.moma.rs",
  userVerification: "required",
  timeout: 60000
}

```

Листинг 7: Опције за церемонију пријаве.

Провера обе церемоније почиње од података клијента. Тип церемоније и порекло морају одговарати ономе што сервер очекује. Изазов се проверава заједно са колачићем издатим уз њега, након чега се брише из *Redis*-а. Приликом регистрације уклања се и токен магичне везе. Затим се пореди хеш идентификатора ослањајуће стране. Индикатори присуства и верификације корисника морају бити постављени.

Током регистрације сервер парсира објекат атестације. Индикатор атестираног креденцијала мора бити постављен, јер део записа који он означава садржи идентификатор креденцијала и јавни кључ. Из COSE мапе сервер чита јавни кључ и његов алгоритам, прихватајући само ознаке из табеле 1. Креденцијал се након провере уписује у базу, заједно са алгоритмом, бројачем употребе и *aaguid* вредности [26].

Приликом пријаве сервер по идентификатору тражи креденцијал, а ако га не пронађе, захтев се одбија. Затим се пореди бројач употребе са последњом запамћеном вредношћу. Бројач који није порастао, а уз то није ни нула, указује на клонирани кључ. У том случају сервер брише креденцијал, заједно са свим сесијама које су за њега везане, и обавештава власника о томе. Последња ствар коју сервер проверава јесте потпис над записом аутентификатора и сажетком података клијента, на основу сачуваног јавног кључа за тај креденцијал. Преглед свих провера дат је у табели 5.

Табела 5: Провере које сервер спроводи над церемонијама.

Провера	Церемонија	Шта потврђује
AT y flags	регистрација	запис садржи јавни кључ и идентификатор
alg	регистрација	сервер уме да провери будуће потписе
credentialId	пријава	кључ припада неком налогу
signCount	пријава	кључ није клониран
signature	пријава	одговор потиче од власника приватног кључа
type	обе	одговор припада траженој церемонији
origin	обе	захтев потиче са домена сервиса
challenge	обе	одговор је на овај захтев, из истог прегледача
rpIdHash	обе	аутентификатор је потписао за овај сервис
UP и UV y flags	обе	корисник је био присутан и проверен

5.4 Порекло кључа као фактор одлуке о правима

Порекло кључа Креденац утврђује једном, приликом регистрације. Изјава о атестацији мора бити у *android-key* формату, а потпис сваког сертификата проверава се наредним у ланцу. Последњи се пореди са списком коренских *Google* сертификата, при чему ниједан сертификат у ланцу не сме бити на списку повучених. Ове две листе нису уграђене у сервер, већ их Креденац преузима приликом покретања и ажурира једном дневно. Будући да *Google* може додати нови коренски сертификат, а да се списак повучених кључева константно мења, уграђена копија не би била поуздана.

Ланац, иако исправно потписан до коренског сертификата, не пружа информацију о томе ко је креирао први сертификат у ланцу. Зато се структура *KeyDescription* из одељка 4.5 не чита из првог сертификата, већ из сертификата најближег корену који је садржи. Име пакета у тој структури указује само на назив апликације, што није сигурна потврда њеног идентитета. Због тога се уз име пореди и сажетак сертификата којим је Привезак потписан, а који Креденац унапред зна. Изазов, који је такође уписан у *KeyDescription* структуру, мора бити једнак сажетку података клијента ове регистрације, тако да се ланац издат за неку другу одбија.

Резултат провере атестације је логичка вредност коју база чува заједно са кренцијалом. Ова вредност касније постаје део података о сесији и одређује да ли корисник има приступ управљању датотекама или не. Атестација дакле не одређује да ли се корисник може пријавити, већ искључиво којим функционалностима ће имати приступ.

5.5 Сесије и заштита веб слоја

Након успешне пријаве сервер издаје приступни токен и смешта га у колачић. Приступни токен је у *JWT (JSON Web Token)* формату, који је сачињен од заглавља, тврдњи и потписа. Заглавље наводи алгоритам којим је токен потписан. Тврдње садрже идентификаторе корисника, сесије и употребљеног кључа, време издавања и истека, као и ознаку да ли кључ потиче из Привеска. Ту ознаку одређује провера описана у одељку 5.4. Потпис је *HMAC-SHA-256* вредност над прва два дела, а сва три су записана у *base64url* облику. Токен важи пет минута, а његов изглед дат је у листингу 8.

Колачић са тим токеном шаље прегледач, тако да му скрипта коју страница извршава не може приступити. Прегледач овај колачић шаље једино уз захтеве намењене Креденцу. Уз колачић, сваки захтев који мења податке на серверу мора у свом заглављу садржати и *CSRF (Cross-Site Request Forgery)* токен. Овај токен је *HMAC* вредност идентификатора сесије, тако да га сервер не мора памтити. Страница га чува у радној меморији, а не у складишту прегледача.

```

{
  "sub": "d0a1d0a2d0a3d094d095d0...", // идентификатор корисника
  "sid": "9dd0a2d09820d09fd09ed0...", // идентификатор сесије
  "cid": "91d095d082d0a3d088d0a3...", // идентификатор кључа
  "iat": 1789430400, // време издавања
  "exp": 1789430700, // време истека
  "pzk": true // кључ потиче из Привеска
}

```

Листинг 8: Тврдње у приступном токenu.

Уз приступни токен издаје се и токен за освежавање приступног токена који важи један дан. Његов колачић ограничен је само на путање за аутентификацију, а база чува једино његову НМАС вредност. Свако освежавање поништава стари токен и издаје нови који припада истом ланцу. Поновна употреба већ поништеног токена значи да је до њега дошао нападач, због чега се цео ланац тада брише. Изузетак је кратак период од две секунде одмах након извршене замене токена. Прегледач у том тренутку може послати два захтева за освежавање одједном, на пример када је страница отворена у више језичака. У том случају оба захтева добијају исту нову вредност токена за освежавање.

Пре брисања налога тражи се поновна потврда идентитета постојећим кључем, а токен који из поновне потврде настаје важи два минута. Строга политика садржаја шаље се уз сваки одговор и допушта извршавање скрипти једино са истог порекла, док уграђивање странице Креденца у туђе странице забрањује. Путање које не траже сесију имају ограничење броја захтева. Ово ограничење постоји како би се спречио напад ускраћивања услуге (енг. *Denial of Service*, DoS). Променљиви садржај странице гради се функцијом која текст уписује као текст, а никада као елементе. Ова функција такође брише адресе које почињу са javascript: и сличним шемама. На овај начин спречава се уметање малициозних скрипти (енг. *Cross-Site Scripting*, XSS).

5.6 Подаци у мировању

Све што Креденац уписује на диск шифровано је уз помоћ AES алгоритма у GCM режиму. Уз сваки шифровани податак налази се и колона са одговарајућом IV вредношћу. Приликом шифровања, идентификатор реда у табели се користи као придружени податак. На овај начин се спречава да шифрат који се премести из једног реда у други остане валидан, јер у том случају ознака аутентичности не би одговарала.

Сваки корисник има свој кључ за податке дужине 32 бајта. Тај кључ се не чува у отвореном облику, већ се шифрује главним кључем. Помоћу њега су заштићене белешке, имена датотека и њихов садржај. Адреса е-поште и подаци о последњој

употреби кључа имају засебан кључ намењен за личне податке. Главни кључ постоји једино у датотеци заштићеној Локотом и у радној меморији сервера.

С обзиром да је адреса е-поште шифрована, претрага адреса врши се на основу њене НМАС вредности. Редови са креденцијалима и токенима за освежавање имају и НМАС вредност израчунату над свим колонама. Ова вредност се проверава приликом сваког читања. Тиме се спречава измена кључних података о креденцијалу, као што су бројач употребе, јавни кључ и податак о томе да ли креденцијал потиче из Привеска.

Захваљујући овим мерама заштите, спречава се могућност фалсификовања садржаја базе, а њена копија не даје ништа читљиво. Ако нападач добије и право уписа, једино што може јесте да податке обрише или оштети. Оштећене податке сервер открива приликом њиховог читања и бележи информацију о томе, док клијент види само грешку сервера без детаља о узроку.

5.7 Отпремање и преузимање датотека

Приликом отпремања, Креденац не смешта у меморију датотеке у целости, већ их обрађује проточно (енг. *streaming*). Датотека која се отпрема стиже подељена у делове које сервер чита редом без увида у наредни. Због тога поље са величином датотеке мора стићи пре самог садржаја, како би датотека могла да се упише у складиште објеката. Захтев се одбија у случају да је најављена дужина датотеке већа од 50 MB, ако њен назив садржи више од 255 карактера или ако корисник већ поседује 100 сачуваних датотека.

Ток садржаја приликом отпремања пролази кроз омотач који прекида пренос уколико забележена величина премаши најављену, док се садржај истовремено шифрује. Складишту објеката предаје се најављена величина увећана за 16 бајтова, колико заузима ознака аутентичности. У ред табеле датотека уписује се измерена величина. Име датотеке шифрује се засебно од садржаја, а као придружени податак користи се њен идентификатор. Приликом преузимања дешифрују се и садржај и име датотеке, а ако провера ознаке аутентичности не успе, садржај се сматра оштећеним.

5.8 *Signal* интерфејс

Приступни кључеви једног налога уписани су на два места, у бази сервера и у аутентификатору на уређају. Када се кључ уклони на серверу, аутентификатор о томе не сазнаје ништа, те га и даље нуди приликом пријаве. Корисник тада може да изабере кључ који сервер више не прихвата, што онда узрокује грешку. WebAuthn због тога предвиђа три методе којима страница може да јави аутентификатору стање кључева. Прва метода, `signalUnknownCredential`, наводи један кључ који сервер не прихвата.

Друга, `signalAllAcceptedCredentials`, шаље цео списак кључева које налог још увек има. Трећа, `signalCurrentUserDetails`, ажурира име корисника које аутентификатор приказује уз кључ [5].

Страница Креденца ове методе позива када сервер одбије потписани одговор или када корисник обрише кључ у подешавањима. Када сервер не препозна кључ или открије клонирање, страница о томе обавештава аутентификатор. Тај позив не садржи кориснички идентификатор нити остале кључеве повезане са његовим налогом, тако да се нападачу не откривају додатни подаци [5]. На страници са подешавањима корисник је већ верификован, тако да Креденац шаље цео списак валидних кључева. Позив се извршава приликом отварања странице, након уклањања кључа и након брисања налога, при чему списак постаје празан.

Овај API још увек није подржан од стране свих прегледача, те због тога страница грешке занемарује. Успешни позив не потврђује да је менаџер нешто изменио, већ само да су послате опције исправно формиране. Разлог за то јесте да резултат позива не би открио страници које кључеве аутентификатор садржи [5]. Методе и њихове опције приказане су у листингу 9.

```
partial interface PublicKeyCredential {
    Promise signalUnknownCredential(UnknownCredentialOptions);
    Promise signalAllAcceptedCredentials(AllAcceptedCredentialsOptions);
    Promise signalCurrentUserDetails(CurrentUserDetailsOptions);
};

dictionary UnknownCredentialOptions {
    DOMString      rpId;           // идентификатор сервиса
    Base64URLString credentialId; // кључ који сервер не прихвата
};

dictionary AllAcceptedCredentialsOptions {
    DOMString      rpId;
    Base64URLString userId;           // кориснички идентификатор
    sequence<Base64URLString> allAcceptedCredentialIds; // кључеви налога
};

dictionary CurrentUserDetailsOptions {
    DOMString      rpId;
    Base64URLString userId;
    DOMString      name;           // име корисника
    DOMString      displayName;    // име за приказ
};
```

Листинг 9: Методе *Signal* интерфејса.

5.9 Тестирање и ограничења

Тестови Креденца раде над стварним *PostgreSQL* и *Redis* инстанцама, али у засебном простору који се празни пре сваког теста. Захваљујући томе провером су обухваћени и упити базе и рокови важења привремених записа. Обе церемоније спроводи софтверски аутентификатор, који чува P-256 кључ, пријављује бројач употребе и обавља потписивање. Уз њега постоји и тестни ланац атестације, па се и одлука о пореклу кључа проверава без телефона. Тестови покривају обе церемоније, откривање клонирања, ограниченост редова на одговарајуће налоге, откривање неовлашћене измене реда, ланце токена за освежавање и ограничење приступа датотекама. Засебну целину чине провере читања CBOR-а, записа аутентификатора и base64url вредности.

Креденац бележи догађаје од значаја за безбедност, као што су откривено клонирање и поновна употреба потрошеног токена. Ти записи се уписују на стандардни излаз, одакле их *Docker* смешта у датотеке на диску. Те датотеке нису заштићене од измена, па их оператер машине може променити или обрисати. Због тога записи служе за увид у рад система, али не и као доказ о томе шта се догодило.

Главни кључ, НМАС тајна и кључ за личне податке читају се приликом подизања система. Поступак за њихову замену након што подаци већ постоје у бази није имплементиран. Замена би тражила поновно шифровање сваког реда и ново рачунање сваке ознаке аутентичности. Систем је грађен уз претпоставку да се једном изабрани кључеви не мењају.

У овом поглављу описан је Локот. Прво је наведен проблем који он решава и шта се од сефа тајни захтева, затим пут од излаза PRF проширења до кључа којим се тај сеф отвара. Потом следе формат сефа и формат документа у ком се тајне налазе, као и шема која тај садржај описује. Затим су описане команде програма и начин на који се тајне испоручују, ослањање на *libfido2* библиотеку и аутентификовање у прегледачу. После тога се налазе библиотека за JVM којом Креденац отвара сеф, а на крају тестирање и ограничења.

6.1 Проблем и захтеви

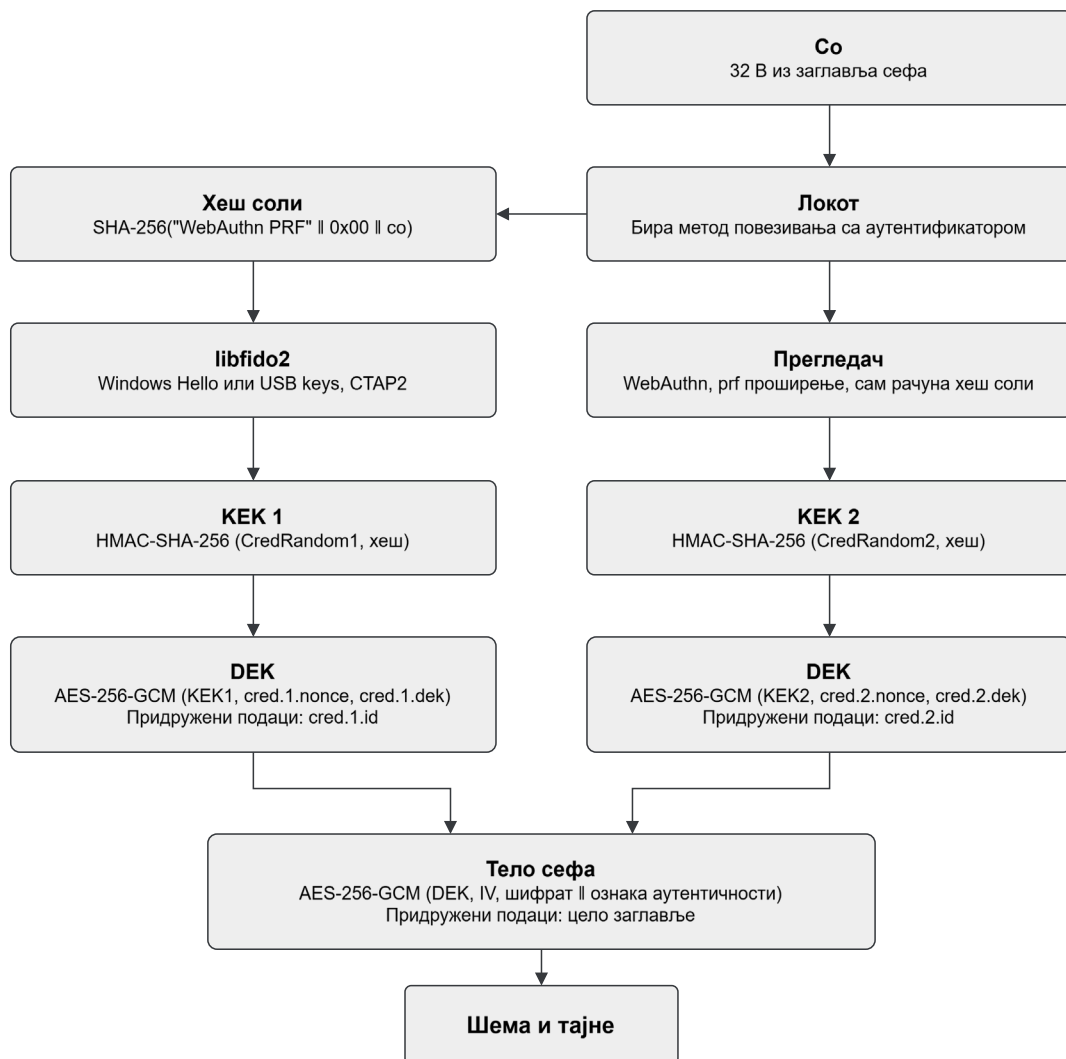
Креденцу су приликом покретања потребне тајне вредности, као што су лозинке база, главни кључ и приватни кључеви за TLS. Уобичајено решење за ово је датотека са променљивама окружења (енг. *environment variables*) коју *Docker* учитава приликом подизања контејнера. Њен садржај би се онда налазио у отвореном облику, што би омогућило сваком ко има приступ диску да га прочита. *Docker* у документацији наводи да предаја тајни кроз променљиве окружења носи ризик од ненамерног откривања, с обзиром на то да су доступне свим процесима и да такође могу доспети у записима [42]. Уместо тога препоручује се да сваки сервис прима само оне тајне које су му потребне, и то у облику датотеке [42].

Локот полази од тога да датотека са тајнама на диск сме да се уписује искључиво у шифрованом облику. Таква датотека онда може да стоји уз изворни код, јер без одговарајућег кључа из ње тајне није могуће прочитати. Сам кључ не сме да се налази на машини, иначе би проблем само променио облик. Сеф треба да подржава отварање са више кључева, како губитак једног не би узроковао губитак тајни.

6.2 Од PRF излаза до кључа сефа

У заглављу сефа налази се у отвореном облику насумична со дужине 32 бајта. Локот ову со користи за формирање улазне вредности PRF проширења, по правилу по ком то чини и прегледач, описаном у одељку 2.6. На овај начин исти кључ враћа исти одговор за ту со без обзира да ли позив потиче из терминала или из прегледача. Тиме је могуће отворити сеф истим кључем на оба начина.

Вредност коју аутентификатор на основу те заједничке соли враћа јесте KEK. Заглавље сефа за сваки уписани кључ садржи један ред са DEK-ом умотаним помоћу KEK-а тог кључа. Током умотавања DEK-а, идентификатор креденцијала учествује као придружени податак. Одмотани DEK користи се за отварање тела сефа. У телу се налазе све тајне, заједно са шемом која их дефинише. Приликом шифровања тела као придружени податак учествује цело заглавље сефа, тако да измена соли или било ког уписаног кључа онемогућава његово отварање.

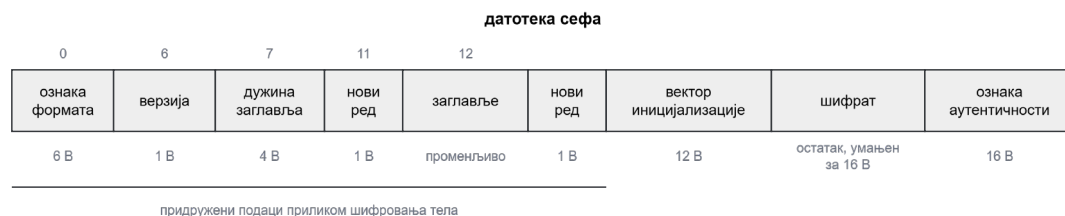


Слика 12: Пут од приступног кључа до тајни у сефу.

Сви уписани кључеви умотавају исти ДЕК, тако да се сеф може отворити било којим од њих. Уписивање новог кључа захтева само још један ред у заглављу, без поновног шифровања целог тела. Сам КЕК се не чува нигде, већ се уместо тога производи приликом сваког отварања сефа, те одмах након употребе брише. Цео пут од приступног кључа до тајни приказан је на слици 12.

6.3 Формат сефа и формат документа са тајнама

Датотека сефа почиње ознаком формата дужине шест бајтова, након које следе број верзије и дужина заглавља. Потом се налази само заглавље, у отвореном облику, након ког су вектор иницијализације, шифрат и ознака аутентичности. Придružени подаци укључују све од почетка датотеке до вектора иницијализације. Приликом сваког уписивања у датотеку бира се нови вектор иницијализације, тако да две верзије истих тајни никада не изгледају исто. Распоред датотеке приказан је на слици 13.



Слика 13: Распоред датотеке сефа.

Заглавље и отворено тело писани су у истом текстуалном формату. Сваки ред садржи име и припадајућу вредност, а раздваја их знак једнакости. Име сме да садржи слова, бројеве, доњу црту, цртицу и тачку. Симбол тарабе означава коментар и текст након ње се игнорише приликом читања. Пример заглавља дат је у листингу 10.

```
project      = kredenac
salt         = d0a1d0a2d0a3d094d095d09dd0a2d098...
cred.0.id    = d09fd09ed091d095d082d0a3d088d0a3
cred.0.rp    = lokot-kredenac.moma.rs
cred.0.nonce = d095d0905354d09552202020
cred.0.dek   = 8e14c0b7532af96d1e0b74c8a25fd930...
cred.1.id    = a25d13e18b8f07523caaf7796c31b10d
cred.1.rp    = lokot-kredenac.moma.rs
cred.1.nonce = 20d09f4fd09145d082d0a3d0
cred.1.dek   = 53545544454e5449504f424544554a55...
```

Листинг 10: Пример заглавља сефа.

Вредност која се простире кроз више редова почиње са три наводника чиме се такође и затвара. Такве вредности потребне су за сертификате и приватне кључеве, који се у сефу чувају као обичне тајне. Свака грешка у документу пријављује се са бројем реда на коме се налази. Пример документа са тајнама дат је у листингу 11.

```

POSTGRES_DB      = kredenac
POSTGRES_USER    = kredenac
POSTGRES_PASSWORD = xQ7mR2vK9pL4nT6wY8sB3dF5gH1jZ0cA
POSTGRES_PORT    = 54312
MASTER_KEY_BASE64 = 9kPq3ZtX1mW7vR0nL5yD8sC2bF4hJ6aU...
CA_CRT           = ""
-----BEGIN CERTIFICATE-----
0KHQotCj0JTQldCd0KLQmCDQn9Ce0JHQLdCC0KPQIncjISEh...
-----END CERTIFICATE-----
""

```

Листинг 11: Пример документа са тајнама.

Док програм ради са тајнама, посматра их као низ карактера, а не као текст. Ово му омогућава да након што заврши са тајном на њено место упише низ нула, што са текстом не би било могуће. Исто важи и за међукорак у ком се бајтови претварају у знакове. Заглавље се чита као обичан текст, пошто се у њему не налазе тајне.

6.4 Шема у TOML запису

За генерисање сефа користи се датотека `lokot.toml`, у којој се дефинише шта сеф садржи. Свака тајна има тип који одређује како се њена вредност генерише. Тип `prompt` означава да вредност, као на пример API кључеве, уноси корисник приликом иницијализације. Тип `random` генерише насумичну вредност са дужином задатом у бајтовима, записану у *base64* или хексадецималном запису. Тип `port` бира број од 49152 до 65535. Ово је динамички опсег који IANA дефинише као нерезервисан [43]. Пример шеме дат је у листингу 12.

Шема је записана у TOML (*Tom's Obvious, Minimal Language*) формату [44], а чита је наменски читач који покрива потребан подскуп. Грешка у синтакси пријављује се заједно са бројем реда, а грешка у декларацији именује тајну на коју се односи. Свака вредност се проверава према свом типу, по броју бајтова и њеном запису, броју знакова или опсегу порта. Издати сертификати и кључеви морају бити у PEM (*Privacy-Enhanced Mail*) запису [45].

Одељак `[certs]` се користи за креирање сертификата са Локотом као издаваоцем. Приликом иницијализације креира се коренски пар кључева. Сваки наведени сертификат конструише се по задатим параметрима и потом потписује тим коренским

кључем. Корен користи RSA кључ величине 4096 бита, док су његови листови дужине 2048 бита и подразумеваног рока важења од десет година. Коренски приватни кључ потписује листове и потом се одбацује без трајног чувања. Настали сертификати и кључеви чувају се као и друге тајне. Овакав ниво ауторитета је довољан, јер се сертификати користе једино за међусобну комуникацију контејнера на истој мрежи.

```
project = "kredenac"

[secrets]
POSTGRES_DB      = { type = "prompt", hint = "database name" }
POSTGRES_PASSWORD = { type = "random", chars = 32 }
POSTGRES_PORT    = { type = "port" }
MASTER_KEY_BASE64 = { type = "random", bytes = 32 }
REDIS_PASSWORD    = { type = "random", chars = 40 }

[certs.redis]
cn = "localhost"
alt = ["DNS:localhost", "DNS:redis", "IP:127.0.0.1"]

[deliver.redis]
REDIS_PASSWORD = { file = "redis.conf", prefix = "requirepass " }
CA_CERT        = "ca.crt"

[compose]
env = ["POSTGRES_PORT"]
```

Листинг 12: Пример TOML шеме Локота.

6.5 Команде и испорука тајни

Рад са сефом одвија се кроз шест команди конзолног програма, приказаних у листингу 13. Свака од ових команди прима одредиште пројекта, било да је то радни директоријум (енг. *working directory*), задата путања или удаљена машина. Команда `init` одбија да се изврши у случају да сеф већ постоји или ако шема недостаје. Команда `rekey` поред кључа за податке такође мења и `co`, па сви уписани кључеви добијају нове умотане DEK копије. Због тога кључеви који се у том процесу не региструју поново бивају уклоњени из сефа.

Команда `unlock` прво брише стари директоријум са тајнама, а потом у њему креира по један поддиректоријум за сваки дефинисани сервис. На *Linux* систему тај директоријум се налази на путањи `/dev/shm`. Ова путања припада систему привремених датотека који се бележи у радној меморији [46]. Тајне се тако никада не уписују на диск, те бивају обрисане сваким гашењем машине. Коренски директоријум је подељен са `0700` дозволом приступа, поддиректоријуми са `0755` и датотеке са `0644`.

употреба: `lokot <команда> [опције]`

<code>init</code>	ствара сеф на основу <code>lokot.toml</code>
<code>unlock</code>	отвара сеф и креира <code>tmpfs</code> датотеке за сервисе
<code>lock</code>	брише датотеке које је <code>unlock</code> креирао
<code>edit</code>	мења вредности у сефу помоћу конзолног уређивача
<code>add-key</code>	додаје нови кључ за отварање сефа
<code>rekey</code>	мења кључеве сефа, задржавајући само тренутно присутне
<code><одредиште></code>	директоријум, или <code>user@host:/путања</code> на другој машини
<code>--rp <id></code>	уз <code>init</code> , <code>add-key</code> и <code>rekey</code> , идентификатор ослањајуће стране

Листинг 13: Команде за управљање сефом.

Уз поддиректоријуме сервиса `unlock` такође ствара и датотеку окружења коју учитава *Docker* током *compose* команде. У њеном заглављу налази се коментар да је датотека генерисана, путања до коренског директоријума са тајнама и вредности наведене у одељку `[compose]` унутар шеме. Команда `lock` уклања коренски директоријум тајни и ову датотеку.

Свака команда која отвара сеф мора најпре да изабере аутентификатор. На *Windows* систему, ако се међу пронађеним уређајима налази *Windows Hello*, подразумевано се користи он. *Windows Hello* је и већ посредник ка свему што систем може да досегне. У осталим случајевима приказује се списак прикључених USB кључева. Ако списак садржи само једну ставку, она аутоматски постаје изабрана.

Одредиште не мора бити рачунар на коме се команда покреће, већ може бити и удаљена машина. У том случају одредиште се задаје у облику `user@host:/путања`, по узору на `scp` команду [47]. Локот тада успоставља `ssh` везу са удаљеном машином и преко ње спроводи SFTP (*SSH File Transfer Protocol*) сесију [48][49]. На тај начин се сеф чита са удаљене машине и касније уписују тајне на њу, док се комуникација са приступним кључем обавља локално.

6.6 Ослањање на изворне библиотеке

Локот се ослања на две изворне библиотеке. Прва је *libfido2*, коју развија и одржава *Yubico* [50]. Локот преко ње шаље СТАР поруке прикљученом кључу и прима његове одговоре. Друга извршава SHA-256 и HMAC функције, шифрује помоћу AES-GCM и издаје сертификате. На *Linux* систему то је системска библиотека *OpenSSL* [51], а на *Windows* систему је то *LibreSSL*, на који се и *libfido2* ослања [52]. Библиотека *libfido2* се на *Linux* систему повезује статички, док се на *Windows* систему обе налазе као динамичке библиотеке уз извршну датотеку Локота.

Приликом уписа новог кључа Локот захтева `hmac-secret` проширење, и то његов `hmac-secret-mc` облик, уведен уз СТАР 2.2. Пошто тај облик враћа излаз већ приликом стварања креденцијала, креирање новог кључа и добијање PRF резултата своди се на једну церемонију. Старији кључеви, који га не подржавају, морају се позвати и други пут како би се након креирања добила PRF вредност.

Приликом читања одговора *libfido2* прихвата само четири облика изјаве о атестацији, док све остале одбија. *Android* од априла 2025. године подразумевано враћа изјаву облика `android-key`, који се не налази међу четири подржана облика [53]. Због тога би сваки упис кључа са телефона резултовао грешком. На *Windows* систему позивалац то не може да избегне, јер *libfido2* на њему увек тражи атестацију. Решење је закрпа која тај назив додаје међу прихваћене облике. Ово је била једина потребна измена зато што `android-key` облик изјаве садржи иста поља као и облик `packed`, који библиотека већ подржава.

6.7 Аутентификација у хибридном режиму транспорта

Библиотека *libfido2* разговара искључиво са кључевима који су прикључени локално, преко USB или NFC везе [49]. Ова библиотека не подржава хибридни режим транспорта, тако да се приступни кључ са телефона не би могао употребити. Ово је решено тиме што је додата опција спровођења церемоније посредством прегледача који подржава хибридни транспорт као што је *Chromium* [54]. Локот зато покреће локални сервер на слободном порту, исписује његову адресу и отвара је у прегледачу.

Адреса подразумевано гласи `http://lokot.localhost:<порт>` из разлога што `localhost` адресе прегледач сматра безбедним, што је предуслов за WebAuthn [55]. С обзиром на то да церемонију води прегледач, корисник може да изабере QR код опцију, те на тај начин користи аутентификатор са свог телефона као што је Привезак.

6.8 Библиотека за JVM и подизање Креденца

Поред конзолног програма, Локот се преводи и у JVM библиотеку. Ова библиотека омогућава читање тајни из сефа, тако што позиваоцу даје копију тражене тајне као низ карактера. Дужност позиваоца је да ту копију обрише чим више није потребна, како би се време током ког се тајна налази у радној меморији свело на минимум. Страница коју Локот користи за аутентификацију путем прегледача такође се као ресурс нуди и кориснику библиотеке за његове потребе откључавања.

Приликом подизања Креденац прво на засебном домену покреће сервер који садржи само део за откључавање тајни. Све путање програмског интерфејса док су тајне недоступне одговарају информацијом да је сервис тренутно недоступан. Главна

нит сервера при томе чека на библиотеку. Када се сеф отвори, сервер за његово откључавање се гаси, а на његово место долази сервер Креденца.

6.9 Тестирање и ограничења

Исправност Локота потврђује се поређењем са унапред познатим тачним вредностима, које за криптографске функције наводе сами стандарди [11][56]. Провере покривају шифровање и хешовање, формат сефа, формат документа, читач TOML записа, правила шеме, уређивач документа тајни и локални сервер. Исти списак тестова покрећу команда `selftest` и аутоматски тестови пројекта.

Губитак свих приступних кључева уписаних у сеф не поседује систем опоравка, јер тајне не постоје нигде изван сефа, а сеф се отвара искључиво кључем. Из тог разлога команда `init` на крају подсећа корисника да у сеф дода и резервни кључ. Упис на удаљено одредиште при томе није атомичан, јер се садржај уписује преко SFTP везе директно у циљну датотеку. Прекид везе може да остави непотпун сеф на удаљеној машини. Локални упис, за разлику од тога, прво попуњава привремену датотеку и тек на крају обавља замену већ потпуне датотеке, тако да прекид оставља стари сеф нетакнут.

Отворене тајне се уписују у *tmpfs* привремени систем датотека, који борави у виртуелној меморији. Такав систем може да заврши у простору за размену, који се налази на диску [46]. Тврдња да тајне не стижу на диск зато важи једино ако је размена страница радне меморије са диском искључена или шифрована.

Глава 7

Закључак

У раду је приказан систем у коме приступни кључеви преузимају улогу коју традиционално имају лозинке. Систем чине три програма, Привезак као менаџер приступних кључева, Креденац као веб апликација и Локот као сеф за серверске тајне. Корисник не памти лозинке, а сервис не мора да их чува и о њима брине.

Први циљ био је приказ целовитог поступка аутентификације приступним кључем. Показано је како кључ настаје у безбедном хардверу телефона, како Привезак саставља одговор аутентификатора и како Креденац тај одговор проверава. Проверене су церемоније регистрације и пријаве, од порекла и изазова до потписа и бројача употребе. Аутентификација се завршава сесијом у прегледачу, коју чине приступни токен и токен за освежавање.

Други циљ био је приказ атестације као основе за одлуку о правима. Описане су обе стране, Привезак који уз кључ прилаже потписану тврдњу о уређају и Креденац који врши проверу до *Google* коренског сертификата. Кључ без потврђеног порекла и даље аутентификује корисника, али му пружа нижи ниво приступа.

Трећи циљ био је употреба приступног кључа у сврху шифровања. Излаз PRF проширења искоришћен је као кључ којим се отвара сеф са серверским тајнама. Локот на тај начин замењује отворену датотеку са тајнама на диску, коју свако на машини може прочитати. Од безбедног хардвера телефона до радне меморије сервера, ни на једном кораку систем тајну не уписује на диск у отвореном облику.

Уз свако од три решења описана су и његова ограничења. Атестација на крају почива на нечему што систем не може да провери. Привезак верује безбедном хардверу телефона, а Креденац листама сертификата које објављује *Google*. То нису пропусти имплементације, већ границе до којих овакав систем досеже.

Даљи рад може да се настави у неколико праваца. Први је замена кључева којима су шифровани већ постојећи подаци у бази. Други је хоризонтално скалирање система, прелазак извршавања са једне машине на више дистрибуираних, уз мерење њихове пропусности. Најзанимљивије питање које остаје јесте процена поузданости самог *Android* уређаја. Атестација показује где је кључ настао, али не и да ли је уређај који га користи исти онај који је тврдњу издао. То питање остаје отворено, али рад је и без одговора на њега показао предности приступних кључева над лозинкама.

Списак слика

Слика 1: Подела FIDO2 радног оквира на WebAuthn и СТАР протоколе.	7
Слика 2: Ток церемоније регистрације.	8
Слика 3: Ток церемоније пријаве.	9
Слика 4: Структура authData записа.	11
Слика 5: Распоред индикатора у бајту заставица.	12
Слика 6: Распоред компоненти система.	18
Слика 7: Ток обраде захтева у Привеску.	22
Слика 8: Хијерархија кључева у Привеску.	23
Слика 9: Настанак атестације и ланац сертификата.	26
Слика 10: Модел података Креденца.	30
Слика 11: Ток регистрације магичном везом.	31
Слика 12: Пут од приступног кључа до тајни у сефу.	40
Слика 13: Распоред датотеке сефа.	41

Списак листинга

Листинг 1: Објекти које WebAuthn церемоније враћају.....	10
Листинг 2: Пример записа clientDataJSON.	11
Листинг 3: Пример COSE записа кључа, приказан у читљивом облику.....	12
Листинг 4: Пример објекта атестације, приказан у читљивом облику.....	13
Листинг 5: Пример PRF захтева и одговора.	27
Листинг 6: Опције за церемонију регистрације.....	32
Листинг 7: Опције за церемонију пријаве.....	33
Листинг 8: Тврдње у приступном токenu.....	35
Листинг 9: Методе <i>Signal</i> интерфејса.	37
Листинг 10: Пример заглавља сефа.	41
Листинг 11: Пример документа са тајнама.	42
Листинг 12: Пример TOML шеме Локота.....	43
Листинг 13: Команде за управљање сефом.....	44

Списак табела

Табела 1: Идентификатори алгоритама у COSE запису кључа.....	13
Табела 2: Претње и мере заштите.	19
Табела 3: Измерена трајања извођења кључа помоћу PBKDF2.....	24
Табела 4: Путање програмског интерфејса.....	29
Табела 5: Провере које сервер спроводи над церемонијама.	33

Списак коришћених скраћеница

Скраћеница	Опис
AAGUID	Authenticator Attestation Globally Unique Identifier (идентификатор произвођача и модела аутентификатора)
AES	Advanced Encryption Standard (напредни стандард за шифровање)
API	Application Programming Interface (апликациони програмски интерфејс)
AT	Attested Credential Data (индикатор присуства података о креденцијалу)
BE	Backup Eligibility (индикатор подобности за резервну копију)
BS	Backup State (индикатор стања резервне копије)
CA	Certificate Authority (издавач сертификата)
CBOR	Concise Binary Object Representation (сажети бинарни запис објеката)
COSE	CBOR Object Signing and Encryption (потписивање и шифровање CBOR објеката)
CSRF	Cross-Site Request Forgery (фалсификовање захтева са друге странице)
CTAP	Client to Authenticator Protocol (протокол између клијента и аутентификатора)
DEK	Data Encryption Key (кључ за шифровање података)
DER	Distinguished Encoding Rules (правила за јединствен бинарни запис)
DoS	Denial of Service (ускраћивање услуге)
ECDSA	Elliptic Curve Digital Signature Algorithm (алгоритам дигиталног потписа над елиптичном кривом)
ED	Extension Data (индикатор присуства резултата проширења)
FIDO	Fast IDentity Online (алијанса за стандардизацију аутентификације)
GCM	Galois/Counter Mode (режим рада блоковске шифре)
HMAC	Hash-based Message Authentication Code (ознака аутентичности заснована на хеш функцији)
IANA	Internet Assigned Numbers Authority (тело које одржава регистре додељених вредности)
IV	Initialization Vector (вектор иницијализације)
JSON	JavaScript Object Notation (формат за размену података)
JVM	Java Virtual Machine (виртуелна машина језика Java)
JWT	JSON Web Token (формат токена заснован на JSON запису)

Скраћеница	Опис
KEK	Key Encryption Key (кључ за шифровање кључа)
OTP	One-Time Password (једнократна лозинка)
PBKDF2	Password-Based Key Derivation Function 2 (функција за извођење кључа из лозинке)
PEM	Privacy-Enhanced Mail (текстуални запис сертификата и кључева)
PII	Personally Identifiable Information (подаци који идентификују корисника)
PIN	Personal Identification Number (лични идентификациони број)
PKI	Public Key Infrastructure (инфраструктура јавних кључева)
PRF	Pseudo-Random Function (псеудо-насумична функција)
QR	Quick Response (дводимензионални бар код)
RAM	Random Access Memory (радна меморија)
RP	Relying Party (ослањајућа страна, сервис који захтева пријаву)
RP ID	Relying Party ID (идентификатор ослањајуће стране)
RSA	Rivest-Shamir-Adleman (алгоритам асиметричне криптографије)
SFTP	SSH File Transfer Protocol (протокол за пренос датотека преко SSH везе)
SHA	Secure Hash Algorithm (породица криптографских хеш функција)
SSH	Secure Shell (протокол за безбедан приступ удаљеној машини)
TEE	Trusted Execution Environment (поуздано извршно окружење)
TLS	Transport Layer Security (протокол за заштиту мрежног саобраћаја)
TOML	Tom's Obvious, Minimal Language (формат за конфигурационе датотеке)
UP	User Present (индикатор присуства корисника)
USB	Universal Serial Bus (универзална серијска магистрала)
UV	User Verified (индикатор верификације корисника)
UX	User Experience (корисничко искуство)
W3C	World Wide Web Consortium (конзорцијум за стандарде интернета)
WebAuthn	Web Authentication (спецификација за аутентификацију на интернету)
XSS	Cross-Site Scripting (уметање малициозних скрипти)

Списак коришћених појмова

Појам	Објашњење
Атестација	Потписана тврдња о томе где је приватни кључ настао и под каквим условима се чува.
Аутентификатор	Уређај или програм који креира кључеве, чува их и користи за потписивање.
Верификација корисника	Провера идентитета корисника коју спроводи сам аутентификатор, биометријом или уносом PIN-а.
Добављач креденцијала	Апликација коју Android систем позива када страница или друга апликација затражи приступни кључ.
Изазов	Насумична вредност коју сервис шаље и која важи само за то извођење церемоније.
Креденцијал	Пар кључева које аутентификатор ствара за један сервис, заједно са припадајућим подацима као што је његов идентификатор.
Магична веза	Једнократна веза послата е-поштом, којом корисник без лозинке долази до свог налога.
Ознака аутентичности	Вредност која потврђује да подаци нису измењени и да потичу од стране која зна кључ.
Ослањајућа страна	Сервис који захтева пријаву и проверава одговор аутентификатора.
Пецање	Напад у ком лажна страница имитира пријаву на постојећи сервис да би прибавила тајну корисника.
Порекло	Шема, домен и порт странице са које захтев потиче, које прегледач сам уписује у податке клијента.
Придружени подаци	Део који се у GCM режиму не шифрује, али улази у рачун ознаке аутентичности.
Приступни кључ	Пар криптографских кључева везан за један сервис, којим се корисник пријављује уместо лозинком. Приватни кључ остаје у аутентификатору и не напушта га.
Проширење	Додатна могућност коју сервис може затражити током церемоније, изван самог потписивања.
Резидентни кључ	Приступни кључ који аутентификатор чува код себе, па пријава може почети без иједног унетог податка.
Сажетак	Излаз хеш функције, низ бајтова утврђене дужине који одговара улазу произвољне дужине.
Сеф	Датотека у којој Локот чува серверске тајне у шифрованом облику.

Појам	Објашњење
Со	Насумична вредност која у функцију улази заједно са тајном, тако да исти улаз даје различите излазе.
Умотавање	Шифровање једног кључа другим кључем, да би први могао да се чува на диску.
Хибридни транспорт	Начин на који аутентификатор са телефона учествује у церемонији покренутој на другом уређају, преко QR кода и Bluetooth везе.
Церемонија	Утврђен ток размене порука између странице, прегледача, аутентификатора и сервиса, приликом регистрације или пријаве.

Биографија

Момир Станишић рођен је у Новом Саду, 4. јуна 2003. године.

Основну школу „Светозар Марковић Тоза“ у Новом Саду завршава 2018. године као ђак генерације. Током похађања основне школе такмичи се из области физике, техничког и информатичког образовања, математике и биологије. Најзначајније успехе остварује на такмичењима из физике, где је на републичком нивоу два пута освојио прво и једном треће место. Учествује и на Српској физичкој олимпијади, где осваја треће место.

Такмичарски дух задржава и током даљег школовања које наставља у електро-техничкој школи „Михајло Пупин“, смер електротехничар информационих технологија. У средњој школи осваја прво место на републичком такмичењу из физике, треће место на републичком такмичењу из програмирања и прву награду на републичком такмичењу из основа електротехнике. Поред такмичења, учествује и на семинарима из примењене физике и електронике у оквиру Истраживачке станице „Петница“. На петничкој конференцији „Корак у науку“ 2020. године презентује свој рад „Дизајнирање аутопилота за стабилизацију авиона коришћењем адаптивног ПИД контролера“. Средњу школу завршава са титулом ђака генерације 2022. године.

Школовање наставља на Факултету техничких наука у Новом Саду, где уписује смер софтверско инжењерство и информационе технологије. Између предавања и полагања испита, 2024. године учествује на републичком такмичењу „Сајбер херој“, где заједно са својим тимом представља ФТН и осваја четврто место. У оквиру студентских протеста доприноси борби као део „Блокадне педале“, возећи бицикл од Новог Сада до Крагујевца, Ниша, Београда, Стразбура, Лознице, Суботице и Шапца. У јулу 2026. године полаже последњи испит предвиђен планом и програмом.

Литература

- [1] A. Langley, A Tour of WebAuthn, 2024, <https://www.imperialviolet.org/tourofwebauthn/tourofwebauthn.html>
- [2] L. Lassak, E. Pan, B. Ur, M. Golla, *Why Aren't We Using Passkeys? Obstacles Companies Face Deploying FIDO2 Passwordless Authentication*, Proceedings of the 33rd USENIX Security Symposium, USENIX Association, 2024, <https://www.usenix.org/system/files/usenixsecurity24-lassak.pdf>
- [3] A. Das, J. Bonneau, M. Caesar, N. Borisov, X. Wang, *The Tangled Web of Password Reuse*, Proceedings of the Network and Distributed System Security Symposium (NDSS), Internet Society, 2014, https://www.ndss-symposium.org/wp-content/uploads/2017/09/06_1_1.pdf
- [4] Cybersecurity and Infrastructure Security Agency (CISA), *Implementing Phishing-Resistant MFA*, 2022, <https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa-508c.pdf>
- [5] T. Cappalli, A. Kumar, E. Lundberg, M. Miller, Pascoe, N. Satragno, *Web Authentication: An API for accessing Public Key Credentials Level 3*, W3C Recommendation, World Wide Web Consortium, 25. август 2026, <https://www.w3.org/TR/webauthn-3/>
- [6] FIDO Alliance, *Client to Authenticator Protocol (CTAP) v2.2*, Proposed Standard, 14. јул 2025, <https://fidoalliance.org/specs/fido-v2.2-ps-20250714/fido-client-to-authenticator-protocol-v2.2-ps-20250714.html>
- [7] FIDO Alliance, W3C, *FIDO Alliance and W3C Achieve Major Standards Milestone in Global Effort Towards Simpler, Stronger Authentication on the Web*, 10. април 2018, <https://fidoalliance.org/fido-alliance-and-w3c-achieve-major-standards-milestone-in-global-effort-towards-simpler-stronger-authentication-on-the-web/>
- [8] W3C, FIDO Alliance, *W3C and FIDO Alliance Finalize Web Standard for Secure, Passwordless Logins*, 4. март 2019, <https://www.w3.org/press-releases/2019/webauthn/>
- [9] T. Scott, Hashing Algorithms and Security, Computerphile, YouTube, <https://youtu.be/b4b8ktEV4Bg>
- [10] M. Pound, Hash Collisions & The Birthday Paradox, Computerphile, YouTube, https://youtu.be/jsrAR-el8_o
- [11] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, август 2015, <https://doi.org/10.6028/NIST.FIPS.180-4>

- [12] M. Pound, SHA: Secure Hashing Algorithm, Computerphile, YouTube, <https://youtu.be/DMtFhACPnTY>
- [13] H. Krawczyk, M. Bellare, R. Canetti, HMAC: Keyed-Hashing for Message Authentication, RFC 2104, Internet Engineering Task Force, фебрыя 1997, <https://www.rfc-editor.org/rfc/rfc2104>
- [14] M. Pound, Securing Stream Ciphers (HMAC), Computerphile, YouTube, <https://youtu.be/wlSG3pEiQdc>
- [15] M. Pound, SHA2 Fatal Flaw? (Hash Length Extension Attack), Computerphile, YouTube, <https://youtu.be/gOIBUe1fjX0>
- [16] K. Moriarty, B. Kaliski, A. Rusch, PKCS #5: Password-Based Cryptography Specification Version 2.1, RFC 8018, Internet Engineering Task Force, жануар 2017, <https://www.rfc-editor.org/rfc/rfc8018>
- [17] National Institute of Standards and Technology, Advanced Encryption Standard (AES), FIPS PUB 197-upd1, маі 2023, <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- [18] M. Pound, AES Explained (Advanced Encryption Standard), Computerphile, YouTube, <https://youtu.be/O4xNJsjtN6E>
- [19] M. Dworkin, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC, NIST Special Publication 800-38D, National Institute of Standards and Technology, новембар 2007, <https://doi.org/10.6028/NIST.SP.800-38D>
- [20] M. Pound, AES GCM (Advanced Encryption Standard in Galois Counter Mode), Computerphile, YouTube, https://youtu.be/-fpVv_T4xwA
- [21] M. Pound, What are Digital Signatures?, Computerphile, YouTube, <https://youtu.be/s22eJ1eVLTU>
- [22] National Institute of Standards and Technology, Digital Signature Standard (DSS), FIPS PUB 186-5, 3. фебрыя 2023, <https://doi.org/10.6028/NIST.FIPS.186-5>
- [23] M. Pound, Elliptic Curves, Computerphile, YouTube, <https://youtu.be/NF1pwjL9-DE>
- [24] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, W. Polk, Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile, RFC 5280, Internet Engineering Task Force, маі 2008, <https://www.rfc-editor.org/rfc/rfc5280>
- [25] S. Josefsson, The Base16, Base32, and Base64 Data Encodings, RFC 4648, Internet Engineering Task Force, октобар 2006, <https://www.rfc-editor.org/rfc/rfc4648>
- [26] passkeydeveloper, Passkey Authenticator AAGUIDs, незвычайны спісак, <https://github.com/passkeydeveloper/passkey-authenticator-aaguids>

- [27] J. Schaad, CBOR Object Signing and Encryption (COSE): Structures and Process, RFC 9052, Internet Engineering Task Force, август 2022, <https://www.rfc-editor.org/rfc/rfc9052>
- [28] C. Bormann, P. Hoffman, Concise Binary Object Representation (CBOR), RFC 8949, Internet Engineering Task Force, децембар 2020, <https://www.rfc-editor.org/rfc/rfc8949>
- [29] Internet Assigned Numbers Authority, CBOR Object Signing and Encryption (COSE), регистар додељених вредности, <https://www.iana.org/assignments/cose>
- [30] M. B. Jones, O. Steele, Fully-Specified Algorithms for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE), RFC 9864, Internet Engineering Task Force, октобар 2025, <https://www.rfc-editor.org/rfc/rfc9864>
- [31] Cloudflare, Cloudflare Tunnel, документација, <https://developers.cloudflare.com/tunnel/>
- [32] Docker Inc., What is Docker?, документација, <https://docs.docker.com/get-started/docker-overview/>
- [33] M. Pound, Transport Layer Security (TLS), Computerphile, YouTube, <https://youtu.be/0TLDTodL7Lc>
- [34] M. Pound, TLS Handshake Explained, Computerphile, YouTube, <https://youtu.be/86cQJ0MMses>
- [35] Google, Integrate Credential Manager with your credential provider solution, документација, <https://developer.android.com/identity/sign-in/credential-provider>
- [36] Google, Android Keystore system, документација, <https://developer.android.com/privacy-and-security/keystore>
- [37] OWASP, Password Storage Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [38] Google, Server-side passkey registration, документација, <https://developers.google.com/identity/passkeys/developer-guides/server-registration>
- [39] Google, Verify hardware-backed key pairs with key attestation, документација, <https://developer.android.com/privacy-and-security/security-key-attestation>
- [40] Android Open Source Project, Key and ID attestation, документација, <https://source.android.com/docs/security/features/keystore/attestation>
- [41] E. Le Guevel, Bypassing Android Hardware Attestation from the Analyst's Chair, Quarkslab, 11. август 2026, <https://blog.quarkslab.com/bypassing-android-hardware-attestation.html>

- [42] Docker Inc., Manage secrets securely in Docker Compose, документација, <https://docs.docker.com/compose/how-tos/use-secrets>
- [43] M. Cotton, L. Eggert, J. Touch, M. Westerlund, S. Cheshire, Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry, RFC 6335, Internet Engineering Task Force, август 2011, <https://www.rfc-editor.org/rfc/rfc6335>
- [44] T. Preston-Werner, P. Gedam, et al., TOML v1.0.0, Tom's Obvious, Minimal Language, 11. јануар 2021, <https://toml.io/en/v1.0.0>
- [45] S. Josefsson, S. Leonard, Textual Encodings of PKIX, PKCS, and CMS Structures, RFC 7468, Internet Engineering Task Force, април 2015, <https://www.rfc-editor.org/rfc/rfc7468>
- [46] Linux man-pages, tmpfs(5), упутство, верзија 6.19, 8. фебруар 2026, <https://man7.org/linux/man-pages/man5/tmpfs.5.html>
- [47] OpenBSD, scp(1), OpenSSH secure file copy, упутство, <https://man.openbsd.org/scp.1>
- [48] T. Ylonen, C. Lonvick, The Secure Shell (SSH) Protocol Architecture, RFC 4251, Internet Engineering Task Force, јануар 2006, <https://www.rfc-editor.org/rfc/rfc4251>
- [49] T. Ylonen, S. Lehtinen, SSH File Transfer Protocol, draft-ietf-secsh-filexfer-02, Internet Engineering Task Force, октобар 2001, <https://datatracker.ietf.org/doc/html/draft-ietf-secsh-filexfer-02>
- [50] Yubico, libfido2, изворни код и документација, <https://github.com/Yubico/libfido2>
- [51] OpenSSL Foundation, OpenSSL Library, документација, <https://openssl-library.org/>
- [52] OpenBSD, LibreSSL, документација, <https://www.libressl.org/>
- [53] C. Brand, Attestation format change for the Android FIDO2 API, Android Developers Blog, 24. септембар 2024, <https://android-developers.googleblog.com/2024/09/attestation-format-change-for-android-fido2-api.html>
- [54] J. L. Zapata, Simplifying WebAuthn feature detection for passkeys, Chrome for Developers, 22. јануар 2025, <https://developer.chrome.com/blog/passkeys-client-capabilities>
- [55] M. West, Secure Contexts, W3C Candidate Recommendation Draft, World Wide Web Consortium, 10. новембар 2023, <https://www.w3.org/TR/secure-contexts/>
- [56] M. Nystrom, Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512, RFC 4231, Internet Engineering Task Force, децембар 2005, <https://www.rfc-editor.org/rfc/rfc4231>